```
if (!String.prototype.padStart) {
  String.prototype.padStart = function (targetLength, padString) {
    const str = String(this);
    const pad = padString || ' ';
    if (str.length >= targetLength) return str;
    const needed = targetLength - str.length;
    return pad.repeat(Math.ceil(needed / pad.length)).slice(0, needed) + str;
  };
}
const { Game } = require('./src/main.js');
const game = new Game();
game.start();
const platform = require('./core/platform.js');
const SceneManager = require('./core/scenes.js');
const Input = require('./core/input.js');
const AudioManager = require('./core/audio.js');
const ArtBank = require('./core/artbank.js');
const storage = require('./core/storage.js');
const AuthScene = require('./scenes/auth.js');
const TitleScene = require('./scenes/title.js');
const DifficultyScene = require('./scenes/difficulty.js');
const StoryScene = require('./scenes/story.js');
const StageScene = require('./scenes/stage.js');
const GameOverScene = require('./scenes/gameover.js');
const StageClearScene = require('./scenes/stageclear.js');
const FakeEndingScene = require('./scenes/fakeending.js');
const LoadingScene = require('./scenes/loading.js');
const DebugStageScene = require('./scenes/debugstage.js');
const TrueEndingScene = require('./scenes/trueending.js');
const MpNameScene = require('./scenes/mpname.js');
const MpRoomScene = require('./scenes/mproom.js');
const AchievementScene = require('./scenes/achievements.js');
const AdScene = require('./scenes/ad.js');
class Game {
  constructor() {
    this.designW = 720;
    this.designH = 1280;
    this.canvas = platform.createCanvas();
    this.ctx = this.canvas.getContext('2d');
    const info = platform.getSystemInfoSync();
    this.windowWidth = info.windowWidth;
    this.windowHeight = info.windowHeight;
    this.scale = Math.min(this.windowWidth / this.designW, this.windowHeight / this.designH);
    this.offsetX = (this.windowWidth - this.designW * this.scale) / 2;
    this.offsetY = (this.windowHeight - this.designH * this.scale) / 2;
    this.input = new Input(this);
    this.audio = new AudioManager();
    this.artBank = new ArtBank();
    this.audio.loadBgms();
    this.sceneManager = new SceneManager(this);
    this.lastTime = 0;
    this.running = false;
    this.sceneManager.register('auth', new AuthScene(this));
    this.sceneManager.register('title', new TitleScene(this));
    this.sceneManager.register('difficulty', new DifficultyScene(this));
```

```
  this.sceneManager.register('story', new StoryScene(this));
  this.sceneManager.register('stage', new StageScene(this));
  this.sceneManager.register('gameover', new GameOverScene(this));
  this.sceneManager.register('stageclear', new StageClearScene(this));
  this.sceneManager.register('fakeending', new FakeEndingScene(this));
  this.sceneManager.register('loading', new LoadingScene(this));
  this.sceneManager.register('debugstage', new DebugStageScene(this));
  this.sceneManager.register('trueending', new TrueEndingScene(this));
  this.sceneManager.register('mpname', new MpNameScene(this));
  this.sceneManager.register('mproom', new MpRoomScene(this));
  this.sceneManager.register('achievement', new AchievementScene(this));
  this.sceneManager.register('ad', new AdScene(this));
  if (typeof window !== 'undefined') {
    window.__cloudspireGame = this;
  }
}
toDesign(clientX, clientY) {
  return {
    x: (clientX - this.offsetX) / this.scale,
    y: (clientY - this.offsetY) / this.scale,
  };
}
go(name, params) {
  this.sceneManager.go(name, params);
}
toTitle() {
  this.go('title');
}
start() {
  if (!this.canvas || !this.ctx) return;
  this.input.onAnyTap = () => this.audio.unlock();
  this.input.start();
  const auth = storage.loadAuth();
  this.go('loading');
  this.running = true;
  this.lastTime = Date.now();
  this.tick();
}
tick() {
  if (!this.running) return;
  const now = Date.now();
  const dt = Math.min(0.05, Math.max(0.001, (now - this.lastTime) / 1000));
  this.lastTime = now;
  this.update(dt);
  this.render();
  requestAnimationFrame(() => this.tick());
}
update(dt) {
  this.sceneManager.update(dt);
}
render() {
  const ctx = this.ctx;
  ctx.setTransform(1, 0, 0, 1, 0, 0);
  ctx.fillStyle = '#05070d';
  ctx.fillRect(0, 0, this.windowWidth, this.windowHeight);
```

```
    ctx.setTransform(this.scale, 0, 0, this.scale, this.offsetX, this.offsetY);
    ctx.save();
    ctx.beginPath();
    ctx.rect(0, 0, this.designW, this.designH);
    ctx.clip();
    this.sceneManager.render(ctx);
    ctx.restore();
  }
}
module.exports = {
  Game,
};
function getWx() {
  return typeof wx !== 'undefined' ? wx : null;
}
const platform = {
  isWeChat() {
    return !!getWx();
  },
  getStorage(key, fallback) {
    const wxApi = getWx();
    if (!wxApi) return fallback;
    try {
      const value = wxApi.getStorageSync(key);
      return value === '' || value === undefined || value === null ? fallback : value;
    } catch (err) {
      return fallback;
    }
  },
  setStorage(key, value) {
    const wxApi = getWx();
    if (!wxApi) return;
    try {
      wxApi.setStorageSync(key, value);
    } catch (err) {
      // Storage quota or privacy errors are non-fatal for a single run.
    }
  },
  removeStorage(key) {
    const wxApi = getWx();
    if (!wxApi) return;
    try {
      wxApi.removeStorageSync(key);
    } catch (err) {
      // ignore
    }
  },
  exitApp() {
    const wxApi = getWx();
    if (wxApi && wxApi.exitMiniProgram) {
      wxApi.exitMiniProgram({});
      return;
    }
    if (typeof location !== 'undefined' && location.reload) {
      location.reload();
```

```
    }
  },
  createCanvas() {
    const wxApi = getWx();
    if (wxApi && wxApi.createCanvas) return wxApi.createCanvas();
    if (typeof document !== 'undefined') {
      const canvas = document.createElement('canvas');
      document.body.appendChild(canvas);
      return canvas;
    }
    return null;
  },
  getSystemInfoSync() {
    const wxApi = getWx();
    if (wxApi && wxApi.getSystemInfoSync) {
      const info = wxApi.getSystemInfoSync();
      return {
        windowWidth: info.windowWidth,
        windowHeight: info.windowHeight,
        pixelRatio: info.pixelRatio || 1,
      };
    }
    return { windowWidth: 720, windowHeight: 1280, pixelRatio: 1 };
  },
  showKeyboard(options) {
    const opts = options || {};
    const wxApi = getWx();
    if (wxApi && wxApi.showKeyboard) {
      if (wxApi.onKeyboardConfirm && opts.onConfirm) wxApi.onKeyboardConfirm((res) => { opts.onConfirm(re
s); });
      if (wxApi.onKeyboardComplete && opts.onComplete) wxApi.onKeyboardComplete((res) => { opts.onComplet
e(res); });
      if (wxApi.onKeyboardInput && opts.onInput) wxApi.onKeyboardInput((res) => { opts.onInput(res); });
      wxApi.showKeyboard(opts);
      return;
    }
    if (typeof window !== 'undefined' && window.__wxKeyboardStub) {
      window.__wxKeyboardStub.show(opts);
    }
  },
  hideKeyboard() {
    const wxApi = getWx();
    if (wxApi && wxApi.hideKeyboard) {
      wxApi.hideKeyboard();
      return;
    }
    if (typeof window !== 'undefined' && window.__wxKeyboardStub) {
      window.__wxKeyboardStub.hide();
    }
  },
  vibrateShort() {
    const wxApi = getWx();
    if (wxApi && wxApi.vibrateShort) {
      wxApi.vibrateShort({ type: 'light' });
    }
```

```
  },
};
module.exports = platform;
class SceneManager {
  constructor(game) {
    this.game = game;
    this.scenes = {};
    this.current = null;
  }
  register(name, scene) {
    this.scenes[name] = scene;
  }
  go(name, params) {
    if (!this.scenes[name]) {
      throw new Error(`Unknown scene: ${name}`);
    }
    if (this.current && this.current.exit) this.current.exit();
    this.current = this.scenes[name];
    this.current.enter(params || {});
  }
  update(dt) {
    if (this.current && this.current.update) this.current.update(dt);
  }
  render(ctx) {
    if (this.current && this.current.render) this.current.render(ctx);
  }
}
module.exports = SceneManager;
class Input {
  constructor(game) {
    this.game = game;
    this.active = null;
    this.touches = {};
    this.taps = [];
    this.enabled = true;
    this.handlers = null;
    this.onAnyTap = null;
  }
  start() {
    const wxApi = typeof wx !== 'undefined' ? wx : null;
    if (!wxApi) return;
    this.handlers = {
      start: (event) => this.handleStart(event),
      move: (event) => this.handleMove(event),
      end: (event) => this.handleEnd(event),
    };
    if (wxApi.onTouchStart) wxApi.onTouchStart(this.handlers.start);
    if (wxApi.onTouchMove) wxApi.onTouchMove(this.handlers.move);
    if (wxApi.onTouchEnd) wxApi.onTouchEnd(this.handlers.end);
    if (wxApi.onTouchCancel) wxApi.onTouchCancel(this.handlers.end);
  }
  stop() {
    const wxApi = typeof wx !== 'undefined' ? wx : null;
    if (!wxApi || !this.handlers) return;
    if (wxApi.offTouchStart) wxApi.offTouchStart(this.handlers.start);
```

```
  if (wxApi.offTouchMove) wxApi.offTouchMove(this.handlers.move);
  if (wxApi.offTouchEnd) wxApi.offTouchEnd(this.handlers.end);
  if (wxApi.offTouchCancel) wxApi.offTouchCancel(this.handlers.end);
  this.handlers = null;
}
toDesign(clientX, clientY) {
  if (this.game) return this.game.toDesign(clientX, clientY);
  return { x: clientX, y: clientY };
}
handleStart(event) {
  if (!this.enabled) return;
  if (this.onAnyTap) this.onAnyTap();
  const changed = event.changedTouches || event.touches || [];
  for (const touch of changed) {
    const point = this.toDesign(touch.clientX, touch.clientY);
    this.touches[touch.identifier] = point;
    this.taps.push({ x: point.x, y: point.y, id: touch.identifier });
    if (!this.active) {
      this.active = { id: touch.identifier, x: point.x, y: point.y };
    }
  }
}
handleMove(event) {
  if (!this.enabled) return;
  const changed = event.changedTouches || event.touches || [];
  for (const touch of changed) {
    const point = this.toDesign(touch.clientX, touch.clientY);
    this.touches[touch.identifier] = point;
    if (this.active && this.active.id === touch.identifier) {
      this.active.x = point.x;
      this.active.y = point.y;
    }
  }
}
handleEnd(event) {
  const changed = event.changedTouches || event.touches || [];
  for (const touch of changed) {
    delete this.touches[touch.identifier];
    if (this.active && this.active.id === touch.identifier) {
      this.active = null;
    }
  }
}
getMoveTarget() {
  return this.active ? { x: this.active.x, y: this.active.y } : null;
}
consumeTap() {
  if (!this.taps.length) return null;
  return this.taps.shift();
}
hasTapIn(rect) {
  const index = this.taps.findIndex(
    (tap) => tap.x >= rect.x && tap.x <= rect.x + rect.w && tap.y >= rect.y && tap.y <= rect.y + rect.h
  );
  if (index === -1) return false;
```

```
    this.taps.splice(index, 1);
    return true;
  }
}
module.exports = Input;
const platform = require('./platform.js');
const SAVE_KEY = 'cloudspire_save_v1';
const AUTH_KEY = 'cloudspire_auth_v1';
function loadSave() {
  return platform.getStorage(SAVE_KEY, null);
}
function saveSave(data) {
  const snapshot = Object.assign({}, data, {
    version: 1,
    savedAt: Date.now(),
  });
  platform.setStorage(SAVE_KEY, snapshot);
  return snapshot;
}
function clearSave() {
  platform.removeStorage(SAVE_KEY);
}
function hasSave() {
  return !!loadSave();
}
function loadAuth() {
  return platform.getStorage(AUTH_KEY, null);
}
function saveAuth(data) {
  platform.setStorage(AUTH_KEY, data);
}
function clearAuth() {
  platform.removeStorage(AUTH_KEY);
}
module.exports = {
  SAVE_KEY,
  AUTH_KEY,
  loadSave,
  saveSave,
  clearSave,
  hasSave,
  loadAuth,
  saveAuth,
  clearAuth,
};
const platform = require('./platform.js');
let sharedContext = null;
function getContext() {
  if (sharedContext) return sharedContext;
  const wxApi = typeof wx !== 'undefined' ? wx : null;
  if (wxApi && wxApi.createWebAudioContext) {
    sharedContext = wxApi.createWebAudioContext();
  } else if (typeof AudioContext !== 'undefined') {
    sharedContext = new AudioContext();
  } else if (typeof webkitAudioContext !== 'undefined') {
```

```
    sharedContext = new webkitAudioContext();
  }
  return sharedContext;
}
function loadBufferTrack(track) {
  fetch(track.src)
    .then((response) => response.arrayBuffer())
    .then((data) => getContext().decodeAudioData(data))
    .then((buffer) => {
      track.buffer = buffer;
      track.ready = true;
      if (track.onReady) track.onReady();
    })
    .catch(() => {
      track.fallback = new Audio(track.src);
      track.fallback.loop = true;
      track.ready = true;
      if (track.onReady) track.onReady();
    });
}
function createAudio(src) {
  const wxApi = typeof wx !== 'undefined' ? wx : null;
  if (wxApi && wxApi.createInnerAudioContext) {
    const inner = wxApi.createInnerAudioContext();
    inner.src = src;
    inner.loop = true;
    inner.onError(() => {});
    const track = { kind: 'inner', audio: inner, ready: false };
    inner.onCanplay = () => {
      track.ready = true;
      if (track.onReady) track.onReady();
    };
    inner.onError = () => {
      track.ready = true;
      if (track.onReady) track.onReady();
    };
    return track;
  }
  if (typeof AudioContext !== 'undefined' || typeof webkitAudioContext !== 'undefined') {
    const track = { kind: 'buffer', src, buffer: null, fallback: null, source: null, gain: null };
    loadBufferTrack(track);
    return track;
  }
  if (typeof Audio !== 'undefined') {
    const audio = new Audio(src);
    audio.loop = true;
    const track = { kind: 'inner', audio, ready: false };
    audio.addEventListener('loadeddata', () => {
      track.ready = true;
      if (track.onReady) track.onReady();
    });
    audio.addEventListener('error', () => {
      track.ready = true;
      if (track.onReady) track.onReady();
    });
```

```
    return track;
  }
  return null;
}
function tone(freq, duration, type, gainValue, when, slideTo) {
  const ctx = getContext();
  if (!ctx) return;
  const start = (when === undefined ? 0 : when) + ctx.currentTime;
  const osc = ctx.createOscillator();
  const gain = ctx.createGain();
  osc.type = type || 'square';
  osc.frequency.setValueAtTime(freq, start);
  if (slideTo) {
    osc.frequency.exponentialRampToValueAtTime(slideTo, start + duration);
  }
  gain.gain.setValueAtTime(gainValue || 0.12, start);
  gain.gain.exponentialRampToValueAtTime(0.0001, start + duration);
  osc.connect(gain);
  gain.connect(ctx.destination);
  osc.start(start);
  osc.stop(start + duration + 0.02);
}
function noise(duration, gainValue, when) {
  const ctx = getContext();
  if (!ctx) return;
  const start = (when === undefined ? 0 : when) + ctx.currentTime;
  const bufferSize = Math.max(1, Math.floor(ctx.sampleRate * duration));
  const buffer = ctx.createBuffer(1, bufferSize, ctx.sampleRate);
  const data = buffer.getChannelData(0);
  for (let i = 0; i < bufferSize; i += 1) {
    data[i] = (Math.random() * 2 - 1) * (1 - i / bufferSize);
  }
  const source = ctx.createBufferSource();
  const gain = ctx.createGain();
  const filter = ctx.createBiquadFilter();
  filter.type = 'lowpass';
  filter.frequency.value = 1400;
  source.buffer = buffer;
  gain.gain.setValueAtTime(gainValue || 0.2, start);
  gain.gain.exponentialRampToValueAtTime(0.0001, start + duration);
  source.connect(filter);
  filter.connect(gain);
  gain.connect(ctx.destination);
  source.start(start);
  source.stop(start + duration + 0.02);
}
class AudioManager {
  constructor() {
    this.started = false;
    this.titleTrack = null;
    this.stageTrack = null;
    this.stage2Track = null;
    this.stage3Track = null;
    this.stage4Track = null;
    this.current = null;
```

```
    this.lightingBuffer = null;
    this.lightingSource = null;
    this.lightingGain = null;
    this.lightingReady = false;
  }
  loadBgms() {
    this.titleTrack = createAudio('assets/audio/title.mp3');
    this.stageTrack = createAudio('assets/audio/stage1.mp3');
    this.stage2Track = createAudio('assets/audio/stage2.mp3');
    this.stage3Track = createAudio('assets/audio/stage3.mp3');
    this.stage4Track = createAudio('assets/audio/stage4.mp3');
    this.boss4Track = createAudio('assets/audio/boss4.m4a');
    this.heTrack = createAudio('assets/audio/he.mp3');
    if (this.titleTrack) this.titleTrack.onReady = () => this.tryStartPending(this.titleTrack);
    if (this.stageTrack) this.stageTrack.onReady = () => this.tryStartPending(this.stageTrack);
    if (this.stage2Track) this.stage2Track.onReady = () => this.tryStartPending(this.stage2Track);
    if (this.stage3Track) this.stage3Track.onReady = () => this.tryStartPending(this.stage3Track);
    if (this.stage4Track) this.stage4Track.onReady = () => this.tryStartPending(this.stage4Track);
    if (this.boss4Track) this.boss4Track.onReady = () => this.tryStartPending(this.boss4Track);
    if (this.heTrack) this.heTrack.onReady = () => this.tryStartPending(this.heTrack);
    this.loadLightingSfx();
  }
  loadLightingSfx(onDone) {
    if (this.lightingReady || this.lightingBuffer) {
      if (onDone) onDone();
      return;
    }
    if (typeof fetch === 'undefined') {
      this.lightingReady = true;
      if (onDone) onDone();
      return;
    }
    fetch('assets/audio/lighting.mp3')
      .then((r) => r.arrayBuffer())
      .then((d) => getContext().decodeAudioData(d))
      .then((buf) => {
        this.lightingBuffer = buf;
        this.lightingReady = true;
        if (onDone) onDone();
      })
      .catch(() => {
        this.lightingReady = true;
        if (onDone) onDone();
      });
  }
  loadAll(onProgress) {
    return new Promise((resolve) => {
      const tracks = [this.titleTrack, this.stageTrack, this.stage2Track, this.stage3Track, this.stage4Tr
ack, this.boss4Track, this.heTrack].filter(Boolean);
      const total = tracks.length + 1;
      const readyTrack = (track) =>
        track.ready ||
        !!track.buffer ||
        (track.kind === 'inner' && track.audio && typeof track.audio.duration === 'number' && track.audio
.duration > 0);
```

```
      const check = () => {
        const loaded = tracks.filter(readyTrack).length + (this.lightingReady || !!this.lightingBuffer ?
1 : 0);
        if (onProgress) onProgress({ loaded, total });
        if (loaded >= total) resolve();
      };
      for (const track of tracks) {
        const original = track.onReady;
        track.onReady = () => {
          if (original) original();
          check();
        };
      }
      this.loadLightingSfx(() => check());
      check();
    });
  }
  unlock() {
    const ctx = getContext();
    if (ctx && ctx.resume) ctx.resume();
    this.started = true;
    this.tryStartPending(this.titleTrack);
    this.tryStartPending(this.stageTrack);
    this.tryStartPending(this.stage2Track);
    this.tryStartPending(this.stage3Track);
    this.tryStartPending(this.stage4Track);
    if (this.current && this.current.kind === 'inner') {
      const inner = this.current.audio || this.current.fallback;
      if (inner && (inner.paused || inner.paused === undefined) && inner.play) {
        inner.play().catch(() => {});
      }
    }
  }
  playBgm(track) {
    if (!track || this.current === track) return;
    this.stopBgm();
    if (track.kind === 'inner') {
      const inner = track.audio || track.fallback;
      if (!inner) return;
      inner.currentTime = 0;
      if (inner.play) inner.play().catch(() => {});
      this.current = track;
      return;
    }
    const ctx = getContext();
    if (!ctx) return;
    if (!track.buffer) {
      track.pending = true;
      this.current = track;
      if (track.fallback) {
        track.fallback.currentTime = 0;
        track.fallback.play().catch(() => {});
      }
      return;
    }
```

```
  if (!this.started) {
    track.pending = true;
    this.current = track;
    return;
  }
  this.startBuffer(track);
}
tryStartPending(track) {
  if (!track || track !== this.current || !track.pending || !this.started) return;
  if (track.kind !== 'buffer' || !track.buffer) return;
  this.startBuffer(track);
}
startBuffer(track) {
  const ctx = getContext();
  if (!ctx || !track.buffer) return;
  track.pending = false;
  track.source = ctx.createBufferSource();
  track.source.buffer = track.buffer;
  track.source.loop = true;
  track.gain = ctx.createGain();
  track.gain.gain.value = 1;
  track.source.connect(track.gain);
  track.gain.connect(ctx.destination);
  track.source.start(0);
  this.current = track;
}
playTitleBgm() {
  this.playBgm(this.titleTrack);
}
playStageBgm() {
  this.playBgm(this.stageTrack);
}
playStage2Bgm() {
  this.playBgm(this.stage2Track);
}
playStage3Bgm() {
  this.playBgm(this.stage3Track);
}
playStage4Bgm() {
  this.playBgm(this.stage4Track);
}
playBoss4Bgm() {
  this.playBgm(this.boss4Track);
}
playHeBgm() {
  this.playBgm(this.heTrack);
}
startBgm() {
  this.playTitleBgm();
}
stopBgm() {
  if (!this.current) return;
  const track = this.current;
  track.pending = false;
  if (track.kind === 'buffer') {
```

```
    if (track.source) {
      try {
        track.source.stop();
      } catch (err) {
        // already stopped
      }
      track.source.disconnect();
      track.source = null;
    }
    if (track.gain) {
      track.gain.disconnect();
      track.gain = null;
    }
  } else {
    const inner = track.audio || track.fallback;
    if (inner && inner.pause) inner.pause();
    if (inner && typeof inner.currentTime === 'number') inner.currentTime = 0;
  }
  this.current = null;
}
shoot() {
  tone(900, 0.05, 'square', 0.13, 0, 620);
}
explosion() {
  noise(0.3, 0.4);
  tone(160, 0.2, 'sawtooth', 0.2, 0, 60);
}
pickup() {
  tone(660, 0.07, 'sine', 0.14);
  tone(990, 0.08, 'sine', 0.14, 0.07);
}
levelup() {
  tone(523, 0.09, 'square', 0.15);
  tone(659, 0.09, 'square', 0.15, 0.08);
  tone(784, 0.14, 'square', 0.15, 0.16);
}
bomb() {
  tone(300, 0.8, 'sawtooth', 0.22, 0, 40);
  noise(0.9, 0.3);
  tone(1200, 0.4, 'sine', 0.1, 0.1, 1600);
}
thunder() {
  noise(1.1, 0.5);
  tone(70, 1.0, 'sawtooth', 0.35, 0, 28);
  tone(140, 0.5, 'square', 0.25, 0.06, 45);
}
playLighting() {
  const ctx = getContext();
  if (!ctx) return;
  this.stopLighting();
  if (this.lightingBuffer) {
    const src = ctx.createBufferSource();
    src.buffer = this.lightingBuffer;
    src.loop = false;
    const gain = ctx.createGain();
```

```
      gain.gain.value = 0.8;
      src.connect(gain);
      gain.connect(ctx.destination);
      src.start(0);
      this.lightingSource = src;
      this.lightingGain = gain;
      src.onended = () => {
        if (this.lightingSource === src) {
          this.lightingSource = null;
          this.lightingGain = null;
        }
      };
    } else {
      this.thunder();
    }
  }
  stopLighting() {
    if (this.lightingSource) {
      try { this.lightingSource.stop(); } catch (e) {}
      try { this.lightingSource.disconnect(); } catch (e) {}
      this.lightingSource = null;
    }
    if (this.lightingGain) {
      try { this.lightingGain.disconnect(); } catch (e) {}
      this.lightingGain = null;
    }
  }
  hit() {
    tone(220, 0.09, 'sawtooth', 0.14, 0, 130);
  }
  bossWarning() {
    tone(392, 0.12, 'square', 0.15);
    tone(392, 0.12, 'square', 0.15, 0.14);
    tone(523, 0.2, 'square', 0.17, 0.28);
  }
  bossPhase() {
    tone(330, 0.1, 'square', 0.15);
    tone(494, 0.18, 'square', 0.15, 0.1);
  }
  gameover() {
    tone(392, 0.25, 'triangle', 0.16);
    tone(311, 0.25, 'triangle', 0.16, 0.24);
    tone(233, 0.45, 'triangle', 0.16, 0.48);
  }
  clear() {
    tone(523, 0.12, 'square', 0.15);
    tone(659, 0.12, 'square', 0.15, 0.1);
    tone(784, 0.12, 'square', 0.15, 0.2);
    tone(1047, 0.3, 'square', 0.17, 0.3);
  }
}
module.exports = AudioManager;
const { loadAssets } = require('./assets.js');
class ArtBank {
  constructor() {
```

```
    this.images = {};
    this.ready = false;
    this.promise = null;
  }
  start() {
    this.load();
  }
  load(onProgress) {
    if (this.promise) return this.promise;
    this.promise = new Promise((resolve) => {
      loadAssets((images) => {
        this.images = images;
        this.ready = true;
        resolve();
      }, onProgress);
    });
    return this.promise;
  }
  get(name) {
    const image = this.images[name];
    return image && image.width > 0 ? image : null;
  }
}
module.exports = ArtBank;
const ASSETS = {
  player: 'assets/images/web/player.png',
  enemy_s: 'assets/images/web/enemy_s.png',
  enemy_m: 'assets/images/web/enemy_m.png',
  enemy_h: 'assets/images/web/enemy_h.png',
  enemy_tuxi: 'assets/images/web/enemy_tuxi.png',
  enemy_hudun: 'assets/images/web/enemy_hudun.png',
  enemy_juji: 'assets/images/web/enemy_juji.png',
  enemy_fengwei_a: 'assets/images/web/enemy_fengwei_a.png',
  enemy_fengwei_b: 'assets/images/web/enemy_fengwei_b.png',
  boss: 'assets/images/web/boss.png',
  boss2: 'assets/images/web/boss2.png',
  boss3: 'assets/images/web/boss3.png',
  bullets: 'assets/images/bullets.png',
  mountain_bg: 'assets/images/mountain_bg.png',
  player_bullet: 'assets/images/player_bullet.png',
  enemy_bullet_small: 'assets/images/enemy_bullet_small.png',
  enemy_bullet_medium: 'assets/images/enemy_bullet_medium.png',
  enemy_bullet_large: 'assets/images/enemy_bullet_large.png',
  boss_bullet_1: 'assets/images/boss_bullet_1.png',
  boss_bullet_2: 'assets/images/boss_bullet_2.png',
  boss_bullet_3: 'assets/images/boss_bullet_3.png',
  boss_bullet_4: 'assets/images/boss_bullet_4.png',
  boss_bullet_5: 'assets/images/boss_bullet_5.png',
  boss_bullet_6: 'assets/images/boss_bullet_6.png',
  boss3_bullet_1: 'assets/images/boss3_bullet_1.png',
  boss3_bullet_2: 'assets/images/boss3_bullet_2.png',
  boss3_bullet_3: 'assets/images/boss3_bullet_3.png',
  boss3_bullet_4: 'assets/images/boss3_bullet_4.png',
  boss3_bullet_5: 'assets/images/boss3_bullet_5.png',
  boss3_bullet_6: 'assets/images/boss3_bullet_6.png',
```

```
  be01: 'assets/images/web/be01.jpg',
  be02: 'assets/images/web/be02.jpg',
  be03: 'assets/images/web/be03.jpg',
  be04: 'assets/images/web/be04.jpg',
  he00: 'assets/images/he00.jpg',
  he01: 'assets/images/he01.jpg',
  he02: 'assets/images/he02.jpg',
  he03: 'assets/images/he03.jpg',
  he04: 'assets/images/he04.jpg',
  he05: 'assets/images/he05.jpg',
  enemy_es: 'assets/images/web/enemy_es.png',
  enemy_em: 'assets/images/web/enemy_em.png',
  enemy_eh: 'assets/images/web/enemy_eh.png',
  enemy_ek: 'assets/images/web/enemy_ek.png',
  enemy_en: 'assets/images/web/enemy_en.png',
  enemy_ed: 'assets/images/web/enemy_ed.png',
  ufo_m: 'assets/images/web/ufo_m.png',
  ufo_l: 'assets/images/web/ufo_l.png',
  ufo_gen: 'assets/images/web/ufo_gen.png',
  meteor_small: 'assets/images/web/meteor_small.png',
  meteor_medium: 'assets/images/web/meteor_medium.png',
  meteor_large: 'assets/images/web/meteor_large.png',
  mothership: 'assets/images/web/mothership.png',
  boss_laser: 'assets/images/boss_laser.png',
};
function loadImage(path) {
  return new Promise((resolve) => {
    const wxApi = typeof wx !== 'undefined' ? wx : null;
    let image;
    if (wxApi && wxApi.createImage) {
      image = wxApi.createImage();
    } else if (typeof Image !== 'undefined') {
      image = new Image();
    } else {
      resolve(null);
      return;
    }
    image.onload = () => resolve(image);
    image.onerror = () => resolve(null);
    image.src = path;
  });
}
async function loadAssets(callback, onProgress) {
  const images = {};
  const entries = Object.entries(ASSETS);
  let loaded = 0;
  await Promise.all(
    entries.map(async ([key, path]) => {
      images[key] = await loadImage(path);
      loaded += 1;
      if (onProgress) onProgress({ loaded, total: entries.length });
    })
  );
  if (callback) callback(images);
  return images;
```

```
}
module.exports = {
  ASSETS,
  loadImage,
  loadAssets,
};
const { TAU, rand, randInt, clamp } = require('../utils/math.js');
class ParticleSystem {
  constructor() {
    this.particles = [];
    this.rings = [];
    this.flash = 0;
  }
  burst(x, y, color, count, speed, life, size) {
    for (let i = 0; i < count; i += 1) {
      const angle = rand(0, TAU);
      const velocity = rand(speed * 0.3, speed);
      this.particles.push({
        x,
        y,
        vx: Math.cos(angle) * velocity,
        vy: Math.sin(angle) * velocity,
        life: rand(life * 0.5, life),
        maxLife: life,
        size: rand(size * 0.5, size),
        color,
        gravity: rand(20, 60),
      });
    }
  }
  sparkle(x, y, color, count) {
    for (let i = 0; i < count; i += 1) {
      const angle = rand(0, TAU);
      const speed = rand(20, 90);
      this.particles.push({
        x,
        y,
        vx: Math.cos(angle) * speed,
        vy: Math.sin(angle) * speed,
        life: rand(0.2, 0.6),
        maxLife: 0.6,
        size: rand(1.5, 3.5),
        color,
        gravity: -10,
        sparkle: true,
      });
    }
  }
  ring(x, y, color, maxRadius, life, width) {
    this.rings.push({
      x,
      y,
      radius: 0,
      maxRadius,
      life,
```

```
      maxLife: life,
      color,
      width: width || 5,
    });
  }
  explosion(x, y, color) {
    this.burst(x, y, color, 26, 260, 0.7, 5);
    this.burst(x, y, '#fff7d9', 12, 180, 0.5, 4);
    this.ring(x, y, color, 90, 0.5, 6);
  }
  update(dt) {
    for (let i = this.particles.length - 1; i >= 0; i -= 1) {
      const p = this.particles[i];
      p.x += p.vx * dt;
      p.y += p.vy * dt;
      p.vy += (p.gravity || 0) * dt;
      p.life -= dt;
      if (p.life <= 0) this.particles.splice(i, 1);
    }
    for (let i = this.rings.length - 1; i >= 0; i -= 1) {
      const r = this.rings[i];
      r.life -= dt;
      r.radius += (r.maxRadius / r.maxLife) * dt;
      if (r.life <= 0) this.rings.splice(i, 1);
    }
    this.flash = Math.max(0, this.flash - dt * 1.2);
  }
  render(ctx) {
    for (const p of this.particles) {
      const alpha = clamp(p.life / p.maxLife, 0, 1);
      ctx.globalAlpha = alpha;
      ctx.fillStyle = p.color;
      if (p.sparkle) {
        ctx.fillRect(p.x - p.size / 2, p.y - p.size / 2, p.size, p.size);
      } else {
        ctx.beginPath();
        ctx.arc(p.x, p.y, p.size, 0, TAU);
        ctx.fill();
      }
    }
    for (const r of this.rings) {
      const alpha = clamp(r.life / r.maxLife, 0, 1);
      ctx.globalAlpha = alpha;
      ctx.strokeStyle = r.color;
      ctx.lineWidth = r.width;
      ctx.beginPath();
      ctx.arc(r.x, r.y, r.radius, 0, TAU);
      ctx.stroke();
    }
    ctx.globalAlpha = 1;
    ctx.lineWidth = 1;
  }
}
module.exports = ParticleSystem;
const platform = require('./platform.js');
```

```
const KEY = 'cloudspire_achievements_v1';
const DEBUG_GATE_KEY = 'cloudspire_debug_unlocked_v1';
const DEFINITIONS = [
  { id: 'pvp', title: 'PVP达人', desc: '在 Local PK 模式取得一次胜利' },
  { id: 'hardClear', title: '虚假的晴空', desc: '达成结局A，首次通关 Hard 模式' },
  { id: 'insaneClear', title: '星海烬余安', desc: '达成结局B，首次通关 Insane 模式' },
  { id: 'debugUnlock', title: 'Hello,World', desc: '解锁内测跳关模式' },
];
function load() {
  const data = platform.getStorage(KEY, null);
  return data && typeof data === 'object' ? data : {};
}
function save(data) {
  platform.setStorage(KEY, data);
}
function isUnlocked(id) {
  return !!load()[id];
}
// Returns true if the achievement was newly unlocked (was locked before).
function unlock(id) {
  if (!DEFINITIONS.some((d) => d.id === id)) return false;
  const data = load();
  if (data[id]) return false;
  data[id] = true;
  save(data);
  return true;
}
function countUnlocked() {
  const data = load();
  return DEFINITIONS.filter((d) => !!data[d.id]).length;
}
// 内测跳关 gate: the debug-stage button is locked until the player watches one ad.
function isDebugUnlocked() {
  return !!platform.getStorage(DEBUG_GATE_KEY, false);
}
function setDebugUnlocked() {
  platform.setStorage(DEBUG_GATE_KEY, true);
}
module.exports = {
  KEY,
  DEFINITIONS,
  load,
  isUnlocked,
  unlock,
  countUnlocked,
  isDebugUnlocked,
  setDebugUnlocked,
};
const { rand, TAU } = require('../utils/math.js');
const PHASE_DURATION = 30;
const TIME_PHASES = [
  {
    skyTop: [214, 168, 124],
    skyMid: [196, 158, 118],
    terrainFar: [184, 142, 102],
```

```
    terrainNear: [176, 126, 86],
  },
  {
    skyTop: [226, 186, 134],
    skyMid: [212, 170, 118],
    terrainFar: [198, 148, 98],
    terrainNear: [192, 132, 84],
  },
  {
    skyTop: [208, 172, 136],
    skyMid: [198, 164, 130],
    terrainFar: [194, 152, 116],
    terrainNear: [188, 138, 98],
  },
];
function mixColor(a, b, t) {
  return [
    Math.round(a[0] + (b[0] - a[0]) * t),
    Math.round(a[1] + (b[1] - a[1]) * t),
    Math.round(a[2] + (b[2] - a[2]) * t),
  ];
}
function scaleColor(color, factor) {
  return color.map((v) => Math.round(Math.min(255, v * factor)));
}
function css(color) {
  return `rgb(${color[0]},${color[1]},${color[2]})`;
}
function makeCanvas(width, height) {
  const wxApi = typeof wx !== 'undefined' ? wx : null;
  if (wxApi && wxApi.createCanvas && typeof document === 'undefined') {
    const c = wxApi.createCanvas();
    c.width = width;
    c.height = height;
    return c;
  }
  const c = document.createElement('canvas');
  c.width = width;
  c.height = height;
  return c;
}
function smoothStep(a, b, t) {
  const x = Math.max(0, Math.min(1, (t - a) / (b - a)));
  return x * x * (3 - 2 * x);
}
function makeMountains(count, minW, maxW, minH, maxH) {
  const mountains = [];
  for (let i = 0; i < count; i += 1) {
    mountains.push({
      x: rand(-40, 720),
      y: rand(-700, 1280),
      w: rand(minW, maxW),
      h: rand(minH, maxH),
      curve: rand(0.15, 0.35),
      seed: Math.floor(rand(0, 100000)),
```

```
      rock: rand(0.4, 1.6),
    });
  }
  return mountains;
}
class MountainBackground {
  constructor(options) {
    const opts = options || {};
    this.fixed = opts.fixed || null;
    this.rain = opts.rain ? this.makeRain() : null;
    this.sea = !!opts.sea;
    this.cloudtop = !!opts.cloudtop;
    if (this.sea) {
      this.makeSea();
      return;
    }
    if (this.cloudtop) {
      this.makeCloudtop();
      return;
    }
    this.terrain = !!opts.terrain;
    if (this.terrain && this.makeTerrain()) {
      return;
    }
    this.terrain = false;
    this.space = !!opts.space;
    if (this.space) {
      this.makeSpace();
      return;
    }
    this.layers = [
      {
        speed: 16,
        alpha: 0.55,
        mountains: makeMountains(8, 150, 280, 160, 380),
      },
      {
        speed: 36,
        alpha: 0.85,
        mountains: makeMountains(7, 130, 260, 220, 480),
      },
      {
        speed: 64,
        alpha: 1,
        mountains: makeMountains(6, 110, 240, 260, 560),
      },
    ];
    this.haze = [];
    for (let i = 0; i < 8; i += 1) {
      this.haze.push({ x: rand(0, 720), y: rand(0, 1280), w: rand(120, 260) });
    }
    this.time = 0;
  }
  makeSpace() {
    this.time = 0;
```

```
this.starLayers = [
  { speed: 18, stars: this.makeStars(55, 0.4, 1.1, 0.35) },
  { speed: 45, stars: this.makeStars(45, 0.7, 1.6, 0.5) },
  { speed: 85, stars: this.makeStars(30, 1.0, 2.2, 0.8) },
];
const nebColors = ['74,42,138', '42,58,154', '122,42,106', '42,74,122'];
this.nebulae = [];
for (let i = 0; i < 5; i += 1) {
  this.nebulae.push({
    x: rand(0, 720),
    y: rand(0, 1280),
    r: rand(210, 330),
    color: nebColors[i % nebColors.length],
    speed: rand(7, 14),
    alpha: rand(0.15, 0.28),
  });
}
this.dustLanes = [];
for (let i = 0; i < 4; i += 1) {
  this.dustLanes.push({
    x: rand(80, 640),
    y: rand(0, 1280),
    r: rand(180, 320),
    alpha: rand(0.24, 0.4),
    speed: rand(5, 12),
    ph: rand(0, TAU),
  });
}
this.debris = [];
for (let i = 0; i < 16; i += 1) {
  this.debris.push({
    x: rand(0, 720),
    y: rand(0, 1280),
    w: rand(8, 40),
    h: rand(5, 15),
    spin: rand(-1.4, 1.4),
    vx: rand(-12, 12),
    vy: rand(22, 55),
    angle: rand(0, TAU),
  });
}
this.celestials = [];
for (let i = 0; i < 4; i += 1) {
  const planet = i % 2 === 0;
  this.celestials.push({
    kind: planet ? 'planet' : 'moon',
    ring: planet && i === 0,
    x: rand(100, 620),
    y: planet ? rand(120, 980) : rand(-420, 260),
    r: planet ? rand(58, 105) : rand(34, 58),
    speed: rand(10, 18),
    phase: rand(0, TAU),
  });
}
this.shooting = null;
```

```
  this.shootTimer = rand(2, 5);
}
makeStars(count, minR, maxR, bright) {
  const list = [];
  const palette = [
    [232, 238, 255],
    [255, 255, 255],
    [255, 234, 196],
    [201, 242, 255],
    [201, 162, 255],
  ];
  for (let i = 0; i < count; i += 1) {
    const c = palette[Math.floor(rand(0, palette.length))];
    list.push({
      x: rand(0, 720),
      y: rand(0, 1280),
      r: rand(minR, maxR),
      phase: rand(0, TAU),
      bright: bright + rand(0, 0.3),
      rgb: c,
      color: `rgb(${c[0]},${c[1]},${c[2]})`,
      glint: Math.random() < 0.25,
      glow: Math.random() < 0.18,
    });
  }
  return list;
}
makeRain() {
  const drops = [];
  for (let i = 0; i < 90; i += 1) {
    drops.push({
      x: rand(0, 720),
      y: rand(0, 1280),
      len: rand(14, 26),
      speed: rand(700, 1000),
      alpha: rand(0.25, 0.5),
    });
  }
  return drops;
}
palette() {
  if (this.fixed) {
    return {
      skyTop: this.fixed.skyTop,
      skyMid: this.fixed.skyMid,
      far: this.fixed.mountains[0],
      mid: this.fixed.mountains[1],
      near: this.fixed.mountains[2],
      haze: this.fixed.haze || this.fixed.skyMid,
    };
  }
  const total = TIME_PHASES.length * PHASE_DURATION;
  const cycle = (this.time % total) / PHASE_DURATION;
  const index = Math.floor(cycle) % TIME_PHASES.length;
  const t = cycle - Math.floor(cycle);
```

```
  const a = TIME_PHASES[index];
  const b = TIME_PHASES[(index + 1) % TIME_PHASES.length];
  const terrainFar = mixColor(a.terrainFar, b.terrainFar, t);
  const terrainNear = mixColor(a.terrainNear, b.terrainNear, t);
  return {
    skyTop: mixColor(a.skyTop, b.skyTop, t),
    skyMid: mixColor(a.skyMid, b.skyMid, t),
    far: mixColor(terrainFar, terrainNear, 0.3),
    mid: scaleColor(mixColor(terrainFar, terrainNear, 0.55), 0.82),
    near: scaleColor(terrainNear, 0.62),
    haze: mixColor(mixColor(a.skyMid, b.skyMid, t), [255, 255, 255], 0.6),
  };
}
update(dt) {
  this.time += dt;
  if (this.sea) {
    this.updateSea(dt);
    return;
  }
  if (this.cloudtop) {
    this.updateCloudtop(dt);
    return;
  }
  if (this.terrain) {
    this.updateTerrain(dt);
    return;
  }
  if (this.space) {
    for (const layer of this.starLayers) {
      for (const star of layer.stars) {
        star.y += layer.speed * dt;
        if (star.y > 1310) {
          star.y = -10;
          star.x = rand(0, 720);
        }
      }
    }
    for (const neb of this.nebulae) {
      neb.y += neb.speed * dt;
      if (neb.y > 1280 + neb.r) {
        neb.y = -neb.r;
        neb.x = rand(0, 720);
      }
    }
    for (const lane of this.dustLanes) {
      lane.y += lane.speed * dt;
      lane.x += Math.sin(this.time * 0.1 + lane.ph) * 2 * dt;
      if (lane.y > 1280 + lane.r) {
        lane.y = -lane.r;
        lane.x = rand(80, 640);
      }
    }
    for (const d of this.debris) {
      d.x += d.vx * dt;
      d.y += d.vy * dt;
```

```
    d.angle += d.spin * dt;
    if (d.y > 1320 || d.x > 760 || d.x < -60) {
      d.x = rand(0, 720);
      d.y = -30;
      d.vy = rand(22, 55);
    }
  }
  for (const c of this.celestials) {
    c.y += c.speed * dt;
    if (c.y > 1380) {
      c.y = -c.r * 2;
      c.x = rand(120, 600);
    }
  }
  if (this.shooting) {
    this.shooting.x += this.shooting.vx * dt;
    this.shooting.y += this.shooting.vy * dt;
    this.shooting.life -= dt;
    if (this.shooting.life <= 0 || this.shooting.y > 700) this.shooting = null;
  } else {
    this.shootTimer -= dt;
    if (this.shootTimer <= 0) {
      this.shooting = {
        x: rand(120, 560),
        y: rand(-90, -30),
        vx: rand(-300, -180),
        vy: rand(420, 560),
        life: rand(0.5, 0.8),
      };
      this.shootTimer = rand(3, 7);
    }
  }
  return;
}
for (const layer of this.layers) {
  for (const mountain of layer.mountains) {
    mountain.y += layer.speed * dt;
    if (mountain.y > 1280 + mountain.h) {
      mountain.y = -mountain.h * 1.6;
      mountain.x = rand(-40, 720);
    }
  }
}
for (const haze of this.haze) {
  haze.y += 42 * dt;
  haze.x += Math.sin(this.time + haze.w) * 3 * dt;
  if (haze.y > 1320) {
    haze.y = -80;
    haze.x = rand(0, 720);
  }
}
if (this.rain) {
  for (const drop of this.rain) {
    drop.y += drop.speed * dt;
    drop.x += 26 * dt;
```

```
      if (drop.y > 1320) {
        drop.y = -20;
        drop.x = rand(0, 720);
      }
    }
  }
}
render(ctx, fx) {
  const effect = fx || {};
  if (this.sea) {
    this.renderSea(ctx, effect);
    return;
  }
  if (this.cloudtop) {
    this.renderCloudtop(ctx, effect);
    return;
  }
  if (this.terrain) {
    this.renderTerrain(ctx);
    return;
  }
  if (this.space) {
    this.renderSpace(ctx);
    return;
  }
  const p = this.palette();
  const sky = ctx.createLinearGradient(0, 0, 0, 1280);
  sky.addColorStop(0, css(p.skyTop));
  sky.addColorStop(0.45, css(p.skyMid));
  sky.addColorStop(0.8, css(p.far));
  sky.addColorStop(1, css(p.near));
  ctx.fillStyle = sky;
  ctx.fillRect(0, 0, 720, 1280);
  const layerColors = [
    mixColor(p.far, [176, 186, 202], 0.42),
    mixColor(p.mid, p.far, 0.1),
    p.near,
  ];
  for (let i = 0; i < this.layers.length; i += 1) {
    const layer = this.layers[i];
    ctx.globalAlpha = layer.alpha;
    for (const mountain of layer.mountains) {
      this.drawMountain(ctx, mountain, layerColors[i], i === 2);
    }
  }
  for (const haze of this.haze) {
    const g = ctx.createRadialGradient(haze.x, haze.y, 4, haze.x, haze.y, haze.w);
    g.addColorStop(0, `rgba(${p.haze[0]},${p.haze[1]},${p.haze[2]},0.16)`);
    g.addColorStop(0.7, `rgba(${p.haze[0]},${p.haze[1]},${p.haze[2]},0.07)`);
    g.addColorStop(1, `rgba(${p.haze[0]},${p.haze[1]},${p.haze[2]},0)`);
    ctx.fillStyle = g;
    ctx.beginPath();
    ctx.ellipse(haze.x, haze.y, haze.w, haze.w * 0.22, 0, 0, Math.PI * 2);
    ctx.fill();
  }
```

```
    if (this.rain) {
      ctx.lineWidth = 1.2;
      ctx.strokeStyle = 'rgba(185,215,238,0.9)';
      for (const drop of this.rain) {
        ctx.globalAlpha = drop.alpha;
        ctx.beginPath();
        ctx.moveTo(drop.x, drop.y);
        ctx.lineTo(drop.x - 5, drop.y + drop.len);
        ctx.stroke();
      }
      ctx.globalAlpha = 1;
    }
    ctx.save();
    for (let i = 0; i < 340; i += 1) {
      const x = (i * 263 + 37) % 720;
      const y = (i * 97) % 1280;
      const light = i % 3 !== 0;
      ctx.globalAlpha = 0.02 + ((i * 7) % 3) * 0.009;
      ctx.fillStyle = light ? '#fff0d4' : '#2b1606';
      ctx.fillRect(x, y, 1 + ((i * 13) % 2), 1);
    }
    ctx.restore();
    const vignette = ctx.createRadialGradient(360, 640, 200, 360, 640, 760);
    vignette.addColorStop(0, 'rgba(20,12,4,0)');
    vignette.addColorStop(1, 'rgba(20,12,4,0.42)');
    ctx.fillStyle = vignette;
    ctx.fillRect(0, 0, 720, 1280);
  }
  renderSpace(ctx) {
    const sky = ctx.createLinearGradient(0, 0, 0, 1280);
    sky.addColorStop(0, '#04020c');
    sky.addColorStop(0.45, '#0b0618');
    sky.addColorStop(0.8, '#140b24');
    sky.addColorStop(1, '#0a0614');
    ctx.fillStyle = sky;
    ctx.fillRect(0, 0, 720, 1280);
    for (let ni = 0; ni < this.nebulae.length; ni += 1) {
      const neb = this.nebulae[ni];
      const puffs = [
        { dx: 0, dy: 0, r: neb.r, a: 0.85 },
        { dx: Math.cos(ni * 2.399) * neb.r * 0.42, dy: -neb.r * 0.18, r: neb.r * 0.62, a: 0.5 },
        { dx: Math.sin(ni * 1.7) * neb.r * 0.5, dy: neb.r * 0.28, r: neb.r * 0.5, a: 0.4 },
      ];
      for (const puff of puffs) {
        const g = ctx.createRadialGradient(neb.x + puff.dx, neb.y + puff.dy, puff.r * 0.08, neb.x + puff.
dx, neb.y + puff.dy, puff.r);
        g.addColorStop(0, `rgba(${neb.color},${(neb.alpha * puff.a).toFixed(3)})`);
        g.addColorStop(0.62, `rgba(${neb.color},${(neb.alpha * puff.a * 0.55).toFixed(3)})`);
        g.addColorStop(1, `rgba(${neb.color},0)`);
        ctx.fillStyle = g;
        ctx.beginPath();
        ctx.arc(neb.x + puff.dx, neb.y + puff.dy, puff.r, 0, TAU);
        ctx.fill();
      }
      const coreColor = ni % 2 === 0 ? '238,232,255' : '255,240,252';
```

```
      const core = ctx.createRadialGradient(neb.x, neb.y, 2, neb.x, neb.y, neb.r * 0.4);
      core.addColorStop(0, `rgba(${coreColor},${(0.22 + neb.alpha * 0.8).toFixed(3)})`);
      core.addColorStop(0.4, `rgba(${coreColor},${(neb.alpha * 0.5).toFixed(3)})`);
      core.addColorStop(1, `rgba(${coreColor},0)`);
      ctx.fillStyle = core;
      ctx.beginPath();
      ctx.arc(neb.x, neb.y, neb.r * 0.4, 0, TAU);
      ctx.fill();
    }
    for (const lane of this.dustLanes) {
      const g = ctx.createRadialGradient(lane.x, lane.y, lane.r * 0.2, lane.x, lane.y, lane.r);
      g.addColorStop(0, `rgba(4,3,12,${(lane.alpha * 0.7).toFixed(3)})`);
      g.addColorStop(0.7, `rgba(8,5,18,${(lane.alpha * 0.3).toFixed(3)})`);
      g.addColorStop(1, 'rgba(8,5,18,0)');
      ctx.fillStyle = g;
      ctx.beginPath();
      ctx.arc(lane.x, lane.y, lane.r, 0, TAU);
      ctx.fill();
    }
    for (const layer of this.starLayers) {
      for (const star of layer.stars) {
        const tw = star.bright * (0.45 + 0.55 * (0.5 + 0.5 * Math.sin(this.time * (1.5 + star.phase) + st
ar.phase)));
        ctx.globalAlpha = Math.min(1, tw);
        if (star.glow && star.r > 1 && star.rgb) {
          const g = ctx.createRadialGradient(star.x, star.y, 0, star.x, star.y, star.r * 5.5);
          g.addColorStop(0, `rgba(${star.rgb[0]},${star.rgb[1]},${star.rgb[2]},${(tw * 0.26).toFixed(3)})
`);
          g.addColorStop(1, `rgba(${star.rgb[0]},${star.rgb[1]},${star.rgb[2]},0)`);
          ctx.fillStyle = g;
          ctx.beginPath();
          ctx.arc(star.x, star.y, star.r * 5.5, 0, TAU);
          ctx.fill();
        }
        ctx.fillStyle = star.color || '#e8ecff';
        ctx.beginPath();
        ctx.arc(star.x, star.y, star.r, 0, TAU);
        ctx.fill();
        if (star.glint && star.r > 1.5) {
          ctx.strokeStyle = star.color || '#ffffff';
          ctx.lineWidth = 0.8;
          ctx.beginPath();
          ctx.moveTo(star.x - star.r * 2.4, star.y);
          ctx.lineTo(star.x + star.r * 2.4, star.y);
          ctx.moveTo(star.x, star.y - star.r * 2.4);
          ctx.lineTo(star.x, star.y + star.r * 2.4);
          ctx.stroke();
        }
      }
    }
    ctx.globalAlpha = 1;
    ctx.save();
    for (let i = 0; i < 240; i += 1) {
      const x = (i * 173 + 53) % 720;
      const y = (i * 109) % 1280;
```

```
    const tone = i % 4;
    ctx.globalAlpha = 0.03 + ((i * 7) % 4) * 0.02;
    if (tone === 0) ctx.fillStyle = '#dfe8ff';
    else if (tone === 1) ctx.fillStyle = '#ffd9f0';
    else if (tone === 2) ctx.fillStyle = '#c9b8ff';
    else ctx.fillStyle = '#9adcff';
    ctx.fillRect(x, y, 1 + ((i * 13) % 2), 1 + ((i * 17) % 2));
  }
  ctx.restore();
  for (const c of this.celestials) {
    ctx.save();
    ctx.translate(c.x, c.y);
    if (c.kind === 'planet') {
      const glow = ctx.createRadialGradient(0, 0, c.r * 0.4, 0, 0, c.r * 1.6);
      glow.addColorStop(0, 'rgba(150,120,240,0.22)');
      glow.addColorStop(0.6, 'rgba(110,80,200,0.08)');
      glow.addColorStop(1, 'rgba(110,80,200,0)');
      ctx.fillStyle = glow;
      ctx.beginPath();
      ctx.arc(0, 0, c.r * 1.6, 0, TAU);
      ctx.fill();
      const body = ctx.createRadialGradient(-c.r * 0.45, -c.r * 0.45, c.r * 0.08, 0, 0, c.r);
      body.addColorStop(0, '#9c88d8');
      body.addColorStop(0.5, '#57468f');
      body.addColorStop(0.85, '#2c2258');
      body.addColorStop(1, '#120c30');
      ctx.fillStyle = body;
      ctx.beginPath();
      ctx.arc(0, 0, c.r, 0, TAU);
      ctx.fill();
      ctx.save();
      ctx.beginPath();
      ctx.arc(0, 0, c.r, 0, TAU);
      ctx.clip();
      ctx.fillStyle = 'rgba(20,12,46,0.16)';
      for (let b = 0; b < 3; b += 1) {
        const by = -c.r * 0.5 + b * c.r * 0.3;
        ctx.fillRect(-c.r, by, c.r * 2, c.r * 0.11);
      }
      const term = ctx.createRadialGradient(c.r * 0.5, 0, c.r * 0.1, c.r * 0.5, 0, c.r * 1.35);
      term.addColorStop(0, 'rgba(6,3,18,0.55)');
      term.addColorStop(0.7, 'rgba(6,3,18,0.2)');
      term.addColorStop(1, 'rgba(6,3,18,0)');
      ctx.fillStyle = term;
      ctx.fillRect(-c.r, -c.r, c.r * 2, c.r * 2);
      ctx.restore();
      if (c.ring) {
        ctx.save();
        ctx.beginPath();
        ctx.arc(0, 0, c.r, 0, TAU);
        ctx.clip();
        ctx.fillStyle = 'rgba(6,3,18,0.42)';
        ctx.beginPath();
        ctx.ellipse(0, 0, c.r * 1.7, c.r * 0.5, -0.35, 0, TAU);
        ctx.fill();
```

```
        ctx.restore();
      }
      ctx.strokeStyle = c.ring ? 'rgba(222,204,255,0.55)' : 'rgba(215,195,255,0.4)';
      ctx.lineWidth = c.ring ? 7 : 3;
      ctx.beginPath();
      ctx.ellipse(0, 0, c.r * 1.7, c.r * 0.5, -0.35, 0, TAU);
      ctx.stroke();
      ctx.strokeStyle = 'rgba(160,130,235,0.35)';
      ctx.lineWidth = c.ring ? 2.4 : 1.2;
      ctx.beginPath();
      ctx.ellipse(0, 0, c.r * 1.82, c.r * 0.54, -0.35, 0, TAU);
      ctx.stroke();
    } else {
      const glow = ctx.createRadialGradient(0, 0, c.r * 0.5, 0, 0, c.r * 1.8);
      glow.addColorStop(0, 'rgba(220,214,255,0.2)');
      glow.addColorStop(1, 'rgba(220,214,255,0)');
      ctx.fillStyle = glow;
      ctx.beginPath();
      ctx.arc(0, 0, c.r * 1.8, 0, TAU);
      ctx.fill();
      const body = ctx.createRadialGradient(-c.r * 0.35, -c.r * 0.35, c.r * 0.1, 0, 0, c.r);
      body.addColorStop(0, '#f0ecfa');
      body.addColorStop(0.6, '#c3bce0');
      body.addColorStop(1, '#8d86b8');
      ctx.fillStyle = body;
      ctx.beginPath();
      ctx.arc(0, 0, c.r, 0, TAU);
      ctx.fill();
      ctx.save();
      ctx.beginPath();
      ctx.arc(0, 0, c.r, 0, TAU);
      ctx.clip();
      const craters = [
        { dx: -0.25, dy: 0.15, r: 0.2 },
        { dx: 0.28, dy: -0.18, r: 0.16 },
        { dx: 0.1, dy: 0.38, r: 0.12 },
      ];
      for (const crater of craters) {
        ctx.fillStyle = 'rgba(60,45,110,0.28)';
        ctx.beginPath();
        ctx.arc(crater.dx * c.r, crater.dy * c.r, crater.r * c.r, 0, TAU);
        ctx.fill();
        ctx.fillStyle = 'rgba(28,18,62,0.3)';
        ctx.beginPath();
        ctx.arc(crater.dx * c.r - crater.r * c.r * 0.25, crater.dy * c.r - crater.r * c.r * 0.2, crater
.r * c.r * 0.72, 0, TAU);
        ctx.fill();
        ctx.strokeStyle = 'rgba(236,230,255,0.6)';
        ctx.lineWidth = 1.2;
        ctx.beginPath();
        ctx.arc(
          crater.dx * c.r + crater.r * c.r * 0.22,
          crater.dy * c.r + crater.r * c.r * 0.18,
          crater.r * c.r * 0.8,
          Math.PI * 0.1,
```

```
        ctx.lineTo(shield.enemy.x + Math.cos(a) * r, shield.enemy.y + Math.sin(a) * r);
        ctx.stroke();
      }
    }
  }
  renderPickup(ctx, pickup) {
    ctx.save();
    ctx.translate(pickup.x, pickup.y);
    const pulse = 1 + Math.sin(this.time * 6) * 0.12;
    ctx.scale(pulse, pulse);
    if (pickup.type === 'shield') {
      ctx.fillStyle = '#2f7fe0';
      ctx.shadowColor = '#4da6ff';
      ctx.shadowBlur = 14;
      ctx.beginPath();
      ctx.arc(0, 0, 13, 0, Math.PI * 2);
      ctx.fill();
      ctx.fillStyle = '#dff0ff';
      ctx.font = 'bold 15px sans-serif';
      ctx.textAlign = 'center';
      ctx.fillText('盾', 0, 5);
    } else if (pickup.type === 'heart') {
      ctx.fillStyle = '#ff6f91';
      ctx.shadowColor = '#ff6f91';
      ctx.shadowBlur = 12;
      ctx.beginPath();
      ctx.moveTo(0, 8);
      ctx.bezierCurveTo(-10, -2, -7, -12, 0, -6);
      ctx.bezierCurveTo(7, -12, 10, -2, 0, 8);
      ctx.fill();
    } else {
      ctx.fillStyle = '#f7c948';
      ctx.shadowColor = '#ffd36b';
      ctx.shadowBlur = 14;
      ctx.beginPath();
      ctx.arc(0, 0, 13, 0, Math.PI * 2);
      ctx.fill();
      ctx.fillStyle = '#fff';
      ctx.font = 'bold 16px sans-serif';
      ctx.textAlign = 'center';
      ctx.fillText('必', 0, 6);
    }
    ctx.restore();
    ctx.shadowBlur = 0;
  }
  renderHud(ctx) {
      if (this.coop && this.player2) {
        drawCoopHud(ctx, { stage: this.stage.id }, this.player, this.player2, this.hostName || 'P1', this
.guestName || 'P2');
      } else {
        drawHud(ctx, this.player, { stage: this.stage.id });
      }
      if (
        this.boss &&
        this.boss.custom === 'twinGuards' &&
```

```
        (this.state === 'miniBossWarning' || this.state === 'miniBoss' || this.state === 'miniBossDeath')
      ) {
        drawTwinBossHud(ctx, this.boss);
      } else if (this.boss && (this.state === 'boss' || this.state === 'bossDeath' || this.state === 'bos
sWarning')) {
        drawBossHud(ctx, this.boss);
      }
    }
  renderIntro(ctx) {
    if (this.state !== 'intro' || this.introTimer <= 0) return;
    const alpha = Math.min(1, this.introTimer / 0.9);
    ctx.globalAlpha = alpha;
    ctx.fillStyle = 'rgba(12,8,4,0.72)';
    ctx.fillRect(0, 470, 720, 130);
    ctx.fillStyle = '#fff8e8';
    ctx.font = 'bold 42px "STKaiti","KaiTi",serif';
    ctx.textAlign = 'center';
    const main = this.stage.title;
    ctx.fillText(main, 360, 545);
    ctx.globalAlpha = 1;
    ctx.textAlign = 'left';
  }
  renderBombFx(ctx) {
    if (this.bombTextTimer > 0) {
      const alpha = Math.min(1, this.bombTextTimer / 0.35);
      ctx.globalAlpha = Math.min(1, alpha * 0.85);
      ctx.fillStyle = 'rgba(8,12,24,0.38)';
      ctx.fillRect(190, 470, 340, 180);
      ctx.globalAlpha = alpha;
      drawInkTitle(ctx, '全域清扫', 360, 560, 76);
      ctx.globalAlpha = 1;
    }
    if (this.screenFlash > 0) {
      ctx.fillStyle = `rgba(255,255,255,${Math.min(0.85, this.screenFlash).toFixed(3)})`;
      ctx.fillRect(0, 0, 720, 1280);
    }
  }
}
module.exports = StageScene;
const { drawButton, drawPanel } = require('../ui/menu.js');
const RESUME_RECT = { x: 210, y: 500, w: 300, h: 62 };
const SAVE_RECT = { x: 210, y: 590, w: 300, h: 62 };
const EXIT_RECT = { x: 210, y: 680, w: 300, h: 62 };
const EXIT_RECT_INV = { x: 210, y: 590, w: 300, h: 62 };
class PauseScene {
  constructor(game) {
    this.game = game;
  }
  update(input, invincible, noSave, pk) {
      const saveDisabled = invincible || !!noSave;
      const tap = input.consumeTap();
      if (!tap) return null;
      if (this.inRect(tap, RESUME_RECT)) return 'resume';
      if (pk) {
        if (this.inRect(tap, EXIT_RECT_INV)) return 'escape';
```

```
        return null;
      }
      if (!saveDisabled && this.inRect(tap, SAVE_RECT)) return 'save-exit';
      if (this.inRect(tap, saveDisabled ? EXIT_RECT_INV : EXIT_RECT)) return 'exit';
      return null;
    }
  inRect(point, rect) {
    return point.x >= rect.x && point.x <= rect.x + rect.w && point.y >= rect.y && point.y <= rect.y + re
ct.h;
  }
  render(ctx, snapshot, invincible, noSave, pk) {
      const saveDisabled = invincible || !!noSave;
      ctx.fillStyle = 'rgba(4,8,18,0.78)';
      ctx.fillRect(0, 0, 720, 1280);
      drawPanel(ctx, { x: 110, y: 350, w: 500, h: 480 });
      ctx.fillStyle = '#f2e7c8';
      ctx.font = 'bold 42px "STKaiti","KaiTi",serif';
      ctx.textAlign = 'center';
      ctx.fillText('暂停', 360, 440);
      ctx.fillStyle = 'rgba(220,232,255,0.8)';
      ctx.font = '17px sans-serif';
      ctx.fillText(`${snapshot.difficulty.toUpperCase()} • Stage ${snapshot.stage} • Score ${snapshot.s
core}`, 360, 478);
      drawButton(ctx, RESUME_RECT, 'Resume');
      if (pk) {
        drawButton(ctx, EXIT_RECT_INV, 'Escape', { color: '#ff9d80' });
      } else if (saveDisabled) {
        drawButton(ctx, EXIT_RECT_INV, 'Exit', { color: '#ffb3c1' });
      } else {
        drawButton(ctx, SAVE_RECT, 'Save and exit');
        drawButton(ctx, EXIT_RECT, 'Exit', { color: '#ffb3c1' });
      }
    }
}
module.exports = PauseScene;
const { drawButton, drawPanel, drawVignette } = require('../ui/menu.js');
const RETRY_RECT = { x: 170, y: 720, w: 380, h: 64 };
const TITLE_RECT = { x: 170, y: 810, w: 380, h: 64 };
class GameOverScene {
  constructor(game) {
    this.game = game;
    this.params = {};
  }
  enter(params) {
    this.game.audio.stopBgm();
    this.params = params || {};
    this.challenge = !!(params && params.challenge);
  }
  update() {
    const tap = this.game.input.consumeTap();
    if (!tap) return;
    if (this.challenge) {
      if (this.inRect(tap, RETRY_RECT)) this.game.go('title');
      return;
    }
```

```
    if (this.inRect(tap, RETRY_RECT)) {
      this.game.go('stage', { mode: 'new', difficulty: this.params.difficulty || 'easy', stage: this.para
ms.stage || 1 });
    } else if (this.inRect(tap, TITLE_RECT)) {
      this.game.go('title');
    }
  }
  inRect(point, rect) {
    return point.x >= rect.x && point.x <= rect.x + rect.w && point.y >= rect.y && point.y <= rect.y + re
ct.h;
  }
  render(ctx) {
    ctx.fillStyle = '#0a0812';
    ctx.fillRect(0, 0, 720, 1280);
    ctx.fillStyle = '#e5486b';
    ctx.font = 'bold 72px sans-serif';
    ctx.textAlign = 'center';
    ctx.shadowColor = '#ff4060';
    ctx.shadowBlur = 24;
    ctx.fillText('GAME OVER', 360, 460);
    ctx.shadowBlur = 0;
    ctx.fillStyle = '#dce6ff';
    ctx.font = '24px sans-serif';
    ctx.fillText(`Score ${String(this.params.score || 0).padStart(7, '0')}`, 360, 540);
    ctx.fillText(`难度 ${String(this.params.difficulty || 'easy').toUpperCase()}`, 360, 590);
    ctx.fillStyle = 'rgba(220,232,255,0.65)';
    ctx.font = '18px sans-serif';
    ctx.fillText(this.challenge ? '挑战失败 · 本局结束' : '境界已经消散，飞戈仍未折返', 360, 650);
    if (this.challenge) {
      drawButton(ctx, RETRY_RECT, 'Title', { color: '#ffb3c1' });
    } else {
      drawButton(ctx, RETRY_RECT, 'Retry Stage');
      drawButton(ctx, TITLE_RECT, 'Title');
    }
    drawVignette(ctx);
  }
}
module.exports = GameOverScene;
const { drawButton, drawPanel } = require('../ui/menu.js');
const { getDifficulty } = require('../config/difficulty.js');
const { getStage } = require('../config/stages.js');
const TOP_RECT = { x: 170, y: 720, w: 380, h: 64 };
const BOTTOM_RECT = { x: 170, y: 810, w: 380, h: 64 };
class StageClearScene {
  constructor(game) {
    this.game = game;
    this.params = {};
    this.stage = 1;
    this.hasNext = false;
  }
  enter(params) {
    this.game.audio.stopBgm();
    this.params = params || {};
    this.stage = this.params.stage || 1;
    this.progress = this.params.progress || null;
```

```
    const difficulty = getDifficulty(this.params.difficulty || 'easy');
    const nextStage = this.stage + 1;
    this.hasNext = nextStage <= difficulty.stageCount && !!getStage(nextStage).enabled;
  }
  update() {
    const tap = this.game.input.consumeTap();
    if (!tap) return;
    if (this.inRect(tap, TOP_RECT)) {
      if (this.hasNext) {
        this.game.audio.pickup();
        this.game.go('story', { difficulty: this.params.difficulty || 'easy', stage: this.stage + 1, prog
ress: this.progress });
      } else {
        this.game.go('title');
      }
      return;
    }
    if (this.inRect(tap, BOTTOM_RECT)) {
      if (this.hasNext) {
        this.game.go('title');
      } else {
        this.game.go('stage', { mode: 'new', difficulty: this.params.difficulty || 'easy', stage: this.st
age });
      }
    }
  }
  inRect(point, rect) {
    return point.x >= rect.x && point.x <= rect.x + rect.w && point.y >= rect.y && point.y <= rect.y + re
ct.h;
  }
  render(ctx) {
    const sky = ctx.createLinearGradient(0, 0, 0, 1280);
    sky.addColorStop(0, '#182438');
    sky.addColorStop(0.5, '#31465f');
    sky.addColorStop(1, '#1a2637');
    ctx.fillStyle = sky;
    ctx.fillRect(0, 0, 720, 1280);
    ctx.fillStyle = '#ffe9b8';
    ctx.font = 'bold 56px "STKaiti","KaiTi",serif';
    ctx.textAlign = 'center';
    ctx.shadowColor = '#ffd36b';
    ctx.shadowBlur = 20;
    ctx.fillText(`Stage ${this.stage} Clear`, 360, 430);
    ctx.shadowBlur = 0;
    ctx.fillStyle = '#e9eefc';
    ctx.font = '26px sans-serif';
    ctx.fillText(`Score ${String(this.params.score || 0).padStart(7, '0')}`, 360, 530);
    ctx.fillStyle = 'rgba(220,232,255,0.75)';
    ctx.font = '18px sans-serif';
    const stage = getStage(this.stage);
    ctx.fillText(`${stage.title} · ${stage.subtitle}`, 360, 600);
    if (this.hasNext) {
      ctx.fillText('云海深处仍有异动，继续追击', 360, 640);
    } else {
      ctx.fillText('天阙重归宁静，云阙飞戈凯旋', 360, 640);
```

```
    }
    if (this.hasNext) {
      drawButton(ctx, TOP_RECT, '下一关');
      drawButton(ctx, BOTTOM_RECT, 'Return to Title');
    } else {
      drawButton(ctx, TOP_RECT, 'Return to Title');
      drawButton(ctx, BOTTOM_RECT, 'Replay Stage');
    }
  }
}
module.exports = StageClearScene;
const achievements = require('../core/achievements.js');
const PAGES = [
  {
    key: 'be01',
    text: '天阙主宰的巨型机体在猛烈炮火下轰然崩碎，暗黑装甲四分五裂，遍布全身的红色裂纹接连爆裂，包裹核心的异形大脑支离破碎。无数金属残骸裹挟金色火光，向着云海四处飘散。',
  },
  {
    key: 'be02',
    text: '破碎的异形大脑残块之中，一点刺目的猩红色光点挣脱残骸的束缚。',
  },
  {
    key: 'be03',
    text: '它没有随碎片一同坠落，反而冲破云层，不断向上加速，刺破大气层。',
  },
  {
    key: 'be04',
    text: '向着深邃漆黑的宇宙深空疾驰飞去，越变越小，最后化为天边一道转瞬即逝的暗红流光。',
  },
];
const END_TITLE = '结局A · 虚假的晴空';
const END_SUBTITLE = '它并未消亡，只是将火种送往更遥远的星海。';
const EXTRA_PAGES = [
  { key: 'he00', text: '你驾驶飞戈 1，毅然调转航向，追入漆黑的宇宙虚空。' },
  { key: 'he00', text: '前方，是未知的星域，被侵染的机械与宇宙残骸，正静静等待着你的到来。' },
];
class FakeEndingScene {
  constructor(game) {
    this.game = game;
    this.images = { be01: null, be02: null, be03: null, be04: null };
    this.ready = false;
    this.page = 0;
    this.chars = 0;
    this.phase = 'story';
    this.timer = 0;
    this.pageFade = 0;
  }
  enter(params) {
    this.params = params || {};
    this.insane = this.params.difficulty === 'insane';
    this.pages = PAGES;
    this.page = 0;
    this.chars = 0;
    this.phase = 'story';
```

```
    this.timer = 0;
    this.pageFade = 0;
    this.images = {
      be01: this.game.artBank.get('be01'),
      be02: this.game.artBank.get('be02'),
      be03: this.game.artBank.get('be03'),
      be04: this.game.artBank.get('be04'),
      he00: this.game.artBank.get('he00'),
    };
    this.ready = true;
    if (this.params.difficulty === 'hard') achievements.unlock('hardClear');
  }
  update(dt) {
    if (!this.ready) return;
    if (this.phase === 'story' || this.phase === 'extra') {
      this.pageFade = Math.min(1, this.pageFade + dt / 0.5);
      const pages = this.phase === 'extra' ? EXTRA_PAGES : this.pages;
      const current = pages[this.page];
      if (this.chars < current.text.length) {
        this.chars = Math.min(current.text.length, this.chars + dt * 26);
      }
      const tap = this.game.input.consumeTap();
      if (!tap) return;
      if (this.chars < current.text.length) {
        this.chars = current.text.length;
        return;
      }
      if (this.page < pages.length - 1) {
        this.page += 1;
        this.chars = 0;
        this.pageFade = 0;
      } else if (this.phase === 'story') {
        if (this.insane) {
          this.phase = 'black';
          this.timer = 0;
        } else {
          this.phase = 'ending';
          this.timer = 0;
        }
      } else {
        this.game.audio.stopBgm();
        this.game.go('stage', {
          mode: 'new',
          difficulty: 'insane',
          stage: 4,
          progress: this.params.progress,
          invincible: this.params.invincible,
        });
      }
      return;
    }
    if (this.phase === 'black') {
      this.timer += dt;
      if (this.timer >= 1.2) {
        this.phase = 'extra';
```

```
      this.page = 0;
      this.chars = 0;
      this.pageFade = 0;
      this.game.audio.stopBgm();
    }
    return;
  }
  this.timer += dt;
  if (this.timer >= 10) {
    this.game.go('title');
  }
}
wrapText(ctx, text, maxWidth) {
  const lines = [];
  let line = '';
  for (const char of text) {
    const test = line + char;
    if (ctx.measureText(test).width > maxWidth && line) {
      lines.push(line);
      line = char;
    } else {
      line = test;
    }
  }
  if (line) lines.push(line);
  return lines;
}
drawImageCover(ctx, img, alpha) {
  if (!img) return;
  ctx.save();
  ctx.globalAlpha = alpha;
  const scale = Math.max(720 / img.width, 1280 / img.height);
  const w = img.width * scale;
  const h = img.height * scale;
  ctx.drawImage(img, (720 - w) / 2, (1280 - h) / 2, w, h);
  ctx.restore();
}
render(ctx) {
  ctx.fillStyle = '#03030a';
  ctx.fillRect(0, 0, 720, 1280);
  if (!this.ready) {
    ctx.fillStyle = 'rgba(255,255,255,0.45)';
    ctx.font = '18px sans-serif';
    ctx.textAlign = 'center';
    ctx.fillText('影像载入中…', 360, 640);
    ctx.textAlign = 'left';
    return;
  }
  if (this.phase === 'story' || this.phase === 'extra') {
    const pages = this.phase === 'extra' ? EXTRA_PAGES : this.pages;
    const current = pages[this.page];
    const img = this.images[current.key];
    this.drawImageCover(ctx, img, this.pageFade);
    ctx.fillStyle = `rgba(5,4,12,${(0.42 * this.pageFade).toFixed(3)})`;
    ctx.fillRect(0, 0, 720, 1280);
```

```
    ctx.fillStyle = 'rgba(6,5,14,0.78)';
    ctx.fillRect(40, 930, 640, 250);
    ctx.strokeStyle = 'rgba(255,220,150,0.35)';
    ctx.lineWidth = 1;
    ctx.strokeRect(40.5, 930.5, 639, 249);
    const visible = current.text.slice(0, Math.ceil(this.chars));
    ctx.fillStyle = '#f6ecd4';
    ctx.font = '24px "STKaiti","KaiTi","SimSun",serif';
    ctx.textAlign = 'left';
    const lines = this.wrapText(ctx, visible, 600);
    lines.forEach((line, index) => {
      ctx.fillText(line, 70, 980 + index * 36);
    });
    ctx.fillStyle = 'rgba(255,255,255,0.55)';
    ctx.font = '16px sans-serif';
    ctx.textAlign = 'center';
    ctx.fillText(`点击继续 ${this.page + 1} / ${pages.length}`, 360, 1240);
    ctx.textAlign = 'left';
    return;
  }
  if (this.phase === 'black') {
    const blackAlpha = Math.min(1, this.timer / 0.9);
    ctx.fillStyle = `rgba(0,0,0,${blackAlpha.toFixed(3)})`;
    ctx.fillRect(0, 0, 720, 1280);
    return;
  }
  const t = this.timer;
  const whiteAlpha = Math.min(1, Math.max(0, t <= 4 ? t / 2 : 1 - (t - 4) / 2));
  const blackAlpha = Math.min(1, Math.max(0, (t - 4) / 2));
  const titleAlpha = t >= 2 && t < 4.6 ? Math.min(1, (t - 2) / 0.5, (4.6 - t) / 0.5) : 0;
  const subtitleAlpha = t >= 6 ? Math.min(1, (t - 6) / 0.8) : 0;
  ctx.fillStyle = `rgba(255,250,235,${whiteAlpha.toFixed(3)})`;
  ctx.fillRect(0, 0, 720, 1280);
  if (titleAlpha > 0) {
    ctx.globalAlpha = titleAlpha;
    ctx.fillStyle = '#2a1538';
    ctx.font = 'bold 44px "STKaiti","KaiTi",serif';
    ctx.textAlign = 'center';
    ctx.shadowColor = '#d8b6ff';
    ctx.shadowBlur = 18;
    ctx.fillText(END_TITLE, 360, 620);
    ctx.shadowBlur = 0;
    ctx.globalAlpha = 1;
  }
  ctx.fillStyle = `rgba(2,2,6,${blackAlpha.toFixed(3)})`;
  ctx.fillRect(0, 0, 720, 1280);
  if (subtitleAlpha > 0) {
    ctx.globalAlpha = subtitleAlpha;
    ctx.fillStyle = '#cfc4dd';
    ctx.font = '24px "STKaiti","KaiTi","SimSun",serif';
    ctx.textAlign = 'center';
    ctx.fillText(END_SUBTITLE, 360, 640);
    ctx.globalAlpha = 1;
  }
  ctx.textAlign = 'left';
```

```
  }
}
FakeEndingScene.PAGES = PAGES;
module.exports = FakeEndingScene;
const achievements = require('../core/achievements.js');
const PAGES = [
  {
    key: 'he01',
    text: '异形母舰最后的猩红核心被彻底击穿，整艘血肉舰船停止所有机能。厚重的舰体从内部开始连锁崩解，无数增生肉质组织、机械炮舱、异形组件瞬间失活。狂暴的炮火与弹幕彻底停息，持续整场战役的压迫感骤然消散，漆黑太空终于回归死寂。',
  },
  {
    key: 'he02',
    text: '曾经试图逃逸的猩红光点，不再具备任何再生、逃逸、寄宿的力量。本源能量在核心爆炸中层层湮灭，红色脉动彻底熄灭、消散、归零。没有残留、没有潜伏、没有后手，这场异形入侵的根源被完全根除，不再有任何未来威胁。',
  },
  {
    key: 'he03',
    text: '漫天漂浮的敌机残骸、破碎舰片、残余感染物质逐渐在真空之中静默飘散。所有被异形侵蚀的空域彻底净化，诡异的血色雾气尽数褪去。遥远星河重新亮起清澈的白光，被污染的宇宙区域恢复原本安宁的秩序。',
  },
  {
    key: 'he04',
    text: '玩家的战机满身弹痕、装甲斑驳，引擎微微发烫，独自悬停在肃清后的战场中央。历经四关绝境鏖战，机身上每一道伤痕都是对抗异形入侵的证明。战机缓缓减速，紧绷全程的战斗姿态彻底松弛，代表战斗正式落幕。',
  },
  {
    key: 'he05',
    text: '深邃星空澄澈明朗，危险彻底终结。战机调转航向，朝向蔚蓝母星的方向平稳返航。战火熄灭、灾厄终结、星海重安。',
  },
];
const END_TITLE = '云海归宁，万劫终熄';
const END_SUBTITLE = '结局B · 星海烬余安';
class TrueEndingScene {
  constructor(game) {
    this.game = game;
    this.images = {};
    this.ready = false;
    this.page = 0;
    this.chars = 0;
    this.phase = 'fadein';
    this.timer = 0;
    this.fadeTimer = 0;
    this.pageFade = 0;
  }
  enter(params) {
    this.params = params || {};
    this.page = 0;
    this.chars = 0;
    this.phase = 'fadein';
    this.timer = 0;
    this.fadeTimer = 0;
```

```
  this.pageFade = 0;
  this.images = {};
  for (const page of PAGES) {
    this.images[page.key] = this.game.artBank ? this.game.artBank.get(page.key) : null;
  }
  this.ready = true;
  if (this.params.difficulty === 'insane') achievements.unlock('insaneClear');
  this.game.audio.playHeBgm();
}
update(dt) {
  if (!this.ready) return;
  if (this.phase === 'fadein') {
    this.fadeTimer += dt;
    if (this.fadeTimer >= 1.2) this.phase = 'story';
    return;
  }
  if (this.phase === 'story') {
    this.pageFade = Math.min(1, this.pageFade + dt / 0.5);
    const current = PAGES[this.page];
    if (this.chars < current.text.length) {
      this.chars = Math.min(current.text.length, this.chars + dt * 26);
    }
    const tap = this.game.input.consumeTap();
    if (!tap) return;
    if (this.chars < current.text.length) {
      this.chars = current.text.length;
      return;
    }
    if (this.page < PAGES.length - 1) {
      this.page += 1;
      this.chars = 0;
      this.pageFade = 0;
    } else {
      this.phase = 'final';
      this.timer = 0;
    }
    return;
  }
  this.timer += dt;
  if (this.timer >= 5.7) {
    this.game.audio.stopBgm();
    this.game.go('title');
  }
}
wrapText(ctx, text, maxWidth) {
  const lines = [];
  let line = '';
  for (const char of text) {
    const test = line + char;
    if (ctx.measureText(test).width > maxWidth && line) {
      lines.push(line);
      line = char;
    } else {
      line = test;
    }
```

```
    }
    if (line) lines.push(line);
    return lines;
  }
  drawImageCover(ctx, img, alpha) {
    if (!img) return;
    ctx.save();
    ctx.globalAlpha = alpha;
    const scale = Math.max(720 / img.width, 1280 / img.height);
    const w = img.width * scale;
    const h = img.height * scale;
    ctx.drawImage(img, (720 - w) / 2, (1280 - h) / 2, w, h);
    ctx.restore();
  }
  render(ctx) {
    ctx.fillStyle = '#03030a';
    ctx.fillRect(0, 0, 720, 1280);
    if (!this.ready) {
      ctx.fillStyle = 'rgba(255,255,255,0.45)';
      ctx.font = '18px sans-serif';
      ctx.textAlign = 'center';
      ctx.fillText('影像载入中…', 360, 640);
      ctx.textAlign = 'left';
      return;
    }
    if (this.phase === 'fadein' || this.phase === 'story') {
      const current = PAGES[this.page];
      const img = this.images[current.key];
      this.drawImageCover(ctx, img, this.pageFade);
      ctx.fillStyle = `rgba(5,4,12,${(0.42 * this.pageFade).toFixed(3)})`;
      ctx.fillRect(0, 0, 720, 1280);
      ctx.fillStyle = 'rgba(6,5,14,0.78)';
      ctx.fillRect(40, 930, 640, 250);
      ctx.strokeStyle = 'rgba(255,220,150,0.35)';
      ctx.lineWidth = 1;
      ctx.strokeRect(40.5, 930.5, 639, 249);
      const visible = current.text.slice(0, Math.ceil(this.chars));
      ctx.fillStyle = '#f6ecd4';
      ctx.font = '24px "STKaiti","KaiTi","SimSun",serif';
      ctx.textAlign = 'left';
      const lines = this.wrapText(ctx, visible, 600);
      lines.forEach((line, index) => {
        ctx.fillText(line, 70, 980 + index * 36);
      });
      ctx.fillStyle = 'rgba(255,255,255,0.55)';
      ctx.font = '16px sans-serif';
      ctx.textAlign = 'center';
      ctx.fillText(`点击继续 ${this.page + 1} / ${PAGES.length}`, 360, 1240);
      ctx.textAlign = 'left';
      if (this.phase === 'fadein') {
        const blackAlpha = Math.max(0, 1 - this.fadeTimer / 1.2);
        ctx.fillStyle = `rgba(0,0,0,${blackAlpha.toFixed(3)})`;
        ctx.fillRect(0, 0, 720, 1280);
      }
      return;
```

```
    }
    const t = this.timer;
    const whiteAlpha = Math.min(1, t / 1.5);
    const titleAlpha = t >= 1.5 && t < 3.8 ? Math.min(1, (t - 1.5) / 0.5, (3.8 - t) / 0.5) : 0;
    const subAlpha = t >= 2.5 ? Math.min(1, (t - 2.5) / 0.5) : 0;
    const darkAlpha = t >= 4.2 ? Math.min(1, (t - 4.2) / 1.5) : 0;
    ctx.fillStyle = `rgba(255,250,235,${whiteAlpha.toFixed(3)})`;
    ctx.fillRect(0, 0, 720, 1280);
    if (titleAlpha > 0) {
      ctx.globalAlpha = titleAlpha;
      ctx.fillStyle = '#2a1538';
      ctx.font = 'bold 46px "STKaiti","KaiTi",serif';
      ctx.textAlign = 'center';
      ctx.shadowColor = '#d8b6ff';
      ctx.shadowBlur = 18;
      ctx.fillText(END_TITLE, 360, 610);
      ctx.shadowBlur = 0;
      ctx.globalAlpha = 1;
    }
    if (subAlpha > 0) {
      ctx.globalAlpha = subAlpha;
      ctx.fillStyle = '#7a4a8c';
      ctx.font = 'bold 28px "STKaiti","KaiTi","SimSun",serif';
      ctx.textAlign = 'center';
      ctx.fillText(END_SUBTITLE, 360, 690);
      ctx.globalAlpha = 1;
    }
    ctx.fillStyle = `rgba(0,0,0,${darkAlpha.toFixed(3)})`;
    ctx.fillRect(0, 0, 720, 1280);
    ctx.textAlign = 'left';
  }
}
TrueEndingScene.PAGES = PAGES;
module.exports = TrueEndingScene;
const achievements = require('../core/achievements.js');
const { drawButton, drawInkTitle } = require('../ui/menu.js');
const BACK_RECT = { x: 24, y: 24, w: 120, h: 48 };
const CARD_X = 70;
const CARD_W = 580;
const CARD_H = 150;
const CARD_YS = [270, 450, 630, 810];
const CARD_ACCENTS = ['#ffd36b', '#8fd8ff', '#ffb45e', '#c9a0ff'];
function roundedRect(ctx, x, y, w, h, r) {
  const radius = Math.min(r, w / 2, h / 2);
  ctx.beginPath();
  ctx.moveTo(x + radius, y);
  ctx.lineTo(x + w - radius, y);
  ctx.arcTo(x + w, y, x + w, y + radius, radius);
  ctx.lineTo(x + w, y + h - radius);
  ctx.arcTo(x + w, y + h, x + w - radius, y + h, radius);
  ctx.lineTo(x + radius, y + h);
  ctx.arcTo(x, y + h, x, y + h - radius, radius);
  ctx.lineTo(x, y + radius);
  ctx.arcTo(x, y, x + radius, y, radius);
  ctx.closePath();
```

```
}
function drawLock(ctx, cx, cy) {
  ctx.save();
  ctx.translate(cx, cy);
  ctx.strokeStyle = 'rgba(220,232,255,0.55)';
  ctx.fillStyle = 'rgba(220,232,255,0.35)';
  ctx.lineWidth = 1.5;
  ctx.beginPath();
  ctx.arc(0, -4, 5, Math.PI, 0);
  ctx.stroke();
  ctx.fillRect(-7, -3, 14, 11);
  ctx.restore();
}
class AchievementScene {
  constructor(game) {
    this.game = game;
    this.time = 0;
  }
  enter() {
    this.time = 0;
    this.game.audio.playTitleBgm();
  }
  update(dt) {
    this.time += dt;
    const tap = this.game.input.consumeTap();
    if (!tap) return;
    if (this.inRect(tap, BACK_RECT)) {
      this.game.audio.pickup();
      this.game.go('title');
    }
  }
  inRect(point, rect) {
    return point.x >= rect.x && point.x <= rect.x + rect.w && point.y >= rect.y && point.y <= rect.y + re
ct.h;
  }
  render(ctx) {
    const sky = ctx.createLinearGradient(0, 0, 0, 1280);
    sky.addColorStop(0, '#0a1220');
    sky.addColorStop(0.5, '#1b2440');
    sky.addColorStop(1, '#0a101c');
    ctx.fillStyle = sky;
    ctx.fillRect(0, 0, 720, 1280);
    drawInkTitle(ctx, '成就', 360, 150, 56);
    const unlocked = achievements.countUnlocked();
    const total = achievements.DEFINITIONS.length;
    ctx.textAlign = 'center';
    ctx.fillStyle = unlocked === total ? '#9fe8b8' : 'rgba(220,232,255,0.75)';
    ctx.font = 'bold 22px sans-serif';
    ctx.fillText(`已达成 ${unlocked} / ${total}`, 360, 220);
    achievements.DEFINITIONS.forEach((def, index) => {
      const done = achievements.isUnlocked(def.id);
      const x = CARD_X;
      const y = CARD_YS[index];
      const accent = CARD_ACCENTS[index % CARD_ACCENTS.length];
      ctx.save();
```

```
      ctx.globalAlpha = done ? 1 : 0.6;
      ctx.fillStyle = done ? 'rgba(16,28,48,0.9)' : 'rgba(12,18,32,0.82)';
      ctx.strokeStyle = done ? accent : 'rgba(220,232,255,0.45)';
      ctx.lineWidth = done ? 2 : 1.2;
      roundedRect(ctx, x, y, CARD_W, CARD_H, 14);
      ctx.fill();
      ctx.stroke();
      const iconCx = x + 46;
      const iconCy = y + CARD_H / 2;
      if (done) {
        ctx.strokeStyle = accent;
        ctx.lineWidth = 2.5;
        ctx.beginPath();
        ctx.arc(iconCx, iconCy, 22, 0, Math.PI * 2);
        ctx.stroke();
        ctx.fillStyle = accent;
        ctx.font = 'bold 26px sans-serif';
        ctx.textAlign = 'center';
        ctx.fillText('  ', iconCx, iconCy + 9);
      } else {
        ctx.strokeStyle = 'rgba(220,232,255,0.6)';
        ctx.lineWidth = 2;
        ctx.beginPath();
        ctx.arc(iconCx, iconCy, 22, 0, Math.PI * 2);
        ctx.stroke();
        drawLock(ctx, iconCx, iconCy);
      }
      ctx.textAlign = 'left';
      ctx.fillStyle = done ? '#fff3d6' : 'rgba(220,232,255,0.85)';
      ctx.font = 'bold 28px sans-serif';
      ctx.fillText(def.title, x + 92, y + 62);
      ctx.fillStyle = done ? 'rgba(210,226,255,0.85)' : 'rgba(200,212,235,0.6)';
      ctx.font = '18px sans-serif';
      ctx.fillText(def.desc, x + 92, y + 106);
      ctx.textAlign = 'right';
      ctx.fillStyle = done ? '#9fe8b8' : 'rgba(220,232,255,0.45)';
      ctx.font = 'bold 20px sans-serif';
      ctx.fillText(done ? '已达成' : '未达成', x + CARD_W - 24, y + CARD_H / 2 + 7);
      ctx.restore();
    });
    ctx.textAlign = 'center';
    ctx.fillStyle = 'rgba(220,232,255,0.5)';
    ctx.font = '16px sans-serif';
    ctx.fillText('达成条件后自动解锁', 360, 1010);
    ctx.textAlign = 'left';
    drawButton(ctx, BACK_RECT, '返回', { font: 'bold 20px sans-serif' });
  }
}
module.exports = AchievementScene;
const { drawButton } = require('../ui/menu.js');
const CONTINUE_RECT = { x: 210, y: 1060, w: 300, h: 64 };
const AD_LINES = [
  '云阙商城 · 星海补给站',
  '舰载武器限时 8 折',
  '扫码领取新机师礼包',
```

```
];
class AdScene {
  constructor(game) {
    this.game = game;
    this.duration = 15;
    this.timer = 15;
    this.done = false;
    this.time = 0;
    this.onDone = null;
  }
  enter(params) {
    const opts = params || {};
    this.duration = Math.max(1, opts.duration || 15);
    this.timer = this.duration;
    this.done = false;
    this.time = 0;
    this.onDone = typeof opts.onDone === 'function' ? opts.onDone : null;
    this.headline = opts.headline || '广告';
  }
  update(dt) {
    this.time += dt;
    if (!this.done) {
      this.timer -= dt;
      if (this.timer <= 0) {
        this.timer = 0;
        this.done = true;
      }
      return;
    }
    const tap = this.game.input.consumeTap();
    if (!tap) return;
    if (this.inRect(tap, CONTINUE_RECT)) {
      this.game.audio.pickup();
      if (this.onDone) {
        const done = this.onDone;
        this.onDone = null;
        done();
      } else {
        this.game.go('title');
      }
    }
  }
  inRect(point, rect) {
    return point.x >= rect.x && point.x <= rect.x + rect.w && point.y >= rect.y && point.y <= rect.y + re
ct.h;
  }
  render(ctx) {
    const sky = ctx.createLinearGradient(0, 0, 0, 1280);
    sky.addColorStop(0, '#1a1030');
    sky.addColorStop(0.55, '#3a1f52');
    sky.addColorStop(1, '#120b20');
    ctx.fillStyle = sky;
    ctx.fillRect(0, 0, 720, 1280);
    ctx.fillStyle = 'rgba(255,220,120,0.16)';
    for (let i = 0; i < 5; i += 1) {
```

```
      const x = (i * 173 + 60) % 720;
      const y = (i * 257 + 80) % 1280;
      ctx.beginPath();
      ctx.arc(x, y, 42 + i * 6, 0, Math.PI * 2);
      ctx.fill();
    }
    ctx.textAlign = 'center';
    ctx.fillStyle = '#ffd36b';
    ctx.font = 'bold 22px sans-serif';
    ctx.fillText('广告', 360, 150);
    ctx.fillStyle = '#fff3e0';
    ctx.font = 'bold 44px "STKaiti","KaiTi","SimSun",serif';
    ctx.fillText('云阙飞戈 · 星海补给站', 360, 260);
    ctx.fillStyle = '#f2e7ff';
    ctx.font = 'bold 26px sans-serif';
    AD_LINES.forEach((line, index) => {
      ctx.fillText(line, 360, 400 + index * 52);
    });
    const seconds = Math.max(0, Math.ceil(this.timer));
    ctx.fillStyle = 'rgba(255,255,255,0.85)';
    ctx.font = 'bold 26px sans-serif';
    ctx.fillText(this.done ? '广告播放完毕' : `广告剩余 ${seconds} 秒`, 360, 640);
    const barX = 170;
    const barW = 380;
    const progress = this.done ? 1 : 1 - this.timer / this.duration;
    ctx.fillStyle = 'rgba(255,255,255,0.14)';
    ctx.fillRect(barX, 680, barW, 12);
    ctx.fillStyle = '#ffd36b';
    ctx.fillRect(barX, 680, barW * Math.max(0, Math.min(1, progress)), 12);
    if (this.done) {
      drawButton(ctx, CONTINUE_RECT, '继续', { font: 'bold 26px sans-serif', pulse: 0.2 + 0.1 * Math.sin(
this.time * 2), accent: 'rgba(140,220,255,0.9)' });
    } else {
      ctx.fillStyle = 'rgba(220,232,255,0.6)';
      ctx.font = '18px sans-serif';
      ctx.fillText('（模拟广告 · 观看完成后即可继续）', 360, 820);
    }
    ctx.textAlign = 'left';
  }
}
module.exports = AdScene;
const platform = require('../core/platform.js');
const { drawButton, drawPanel, drawInkTitle } = require('../ui/menu.js');
const NAME_RECT = { x: 160, y: 520, w: 400, h: 72 };
const CONFIRM_RECT = { x: 210, y: 680, w: 300, h: 64 };
const BACK_RECT = { x: 24, y: 24, w: 120, h: 48 };
class MpNameScene {
  constructor(game) {
    this.game = game;
    this.name = '';
  }
  enter() {
    this.name = '';
    this.game.audio.playTitleBgm();
  }
```

```
  update() {
    const tap = this.game.input.consumeTap();
    if (!tap) return;
    if (this.inRect(tap, NAME_RECT)) {
      this.openKeyboard();
      return;
    }
    if (this.inRect(tap, CONFIRM_RECT)) {
      if (this.name.trim()) {
        this.game.audio.pickup();
        this.game.go('mproom', { name: this.name.trim() });
      }
      return;
    }
    if (this.inRect(tap, BACK_RECT)) {
      this.game.go('title');
    }
  }
  openKeyboard() {
    platform.showKeyboard({
      defaultValue: this.name,
      maxLength: 12,
      confirmType: 'done',
      onConfirm: (res) => {
        this.name = (res && res.value) || '';
        platform.hideKeyboard();
      },
      onComplete: (res) => {
        const v = res && res.value !== undefined ? res.value : '';
        if (v) this.name = v;
        platform.hideKeyboard();
      },
    });
  }
  inRect(point, rect) {
    return point.x >= rect.x && point.x <= rect.x + rect.w && point.y >= rect.y && point.y <= rect.y + re
ct.h;
  }
  render(ctx) {
    const sky = ctx.createLinearGradient(0, 0, 0, 1280);
    sky.addColorStop(0, '#0a1220');
    sky.addColorStop(0.6, '#1a2540');
    sky.addColorStop(1, '#0a101c');
    ctx.fillStyle = sky;
    ctx.fillRect(0, 0, 720, 1280);
    drawInkTitle(ctx, 'Local PK', 360, 220, 52);
    ctx.textAlign = 'center';
    ctx.fillStyle = 'rgba(220,232,255,0.8)';
    ctx.font = '20px sans-serif';
    ctx.fillText('请输入你的玩家名称', 360, 340);
    drawPanel(ctx, { x: 140, y: 500, w: 440, h: 110 });
    ctx.fillStyle = this.name ? '#f2f6ff' : 'rgba(220,232,255,0.45)';
    ctx.font = 'bold 28px sans-serif';
    ctx.fillText(this.name || '点击输入昵称…', 360, 558);
    drawButton(ctx, CONFIRM_RECT, '确认', {
```

```
      font: 'bold 26px sans-serif',
      accent: 'rgba(140,200,255,0.9)',
    });
    drawButton(ctx, BACK_RECT, '返回', { font: 'bold 20px sans-serif' });
    ctx.textAlign = 'left';
  }
}
module.exports = MpNameScene;
const platform = require('../core/platform.js');
const net = require('../net/multiplayer.js');
const { drawButton, drawPanel, drawInkTitle } = require('../ui/menu.js');
const { clamp } = require('../utils/math.js');
const CREATE_RECT = { x: 160, y: 440, w: 400, h: 72 };
const JOIN_RECT = { x: 160, y: 540, w: 400, h: 72 };
const SERVER_RECT = { x: 160, y: 640, w: 400, h: 64 };
const ROOMCODE_RECT = { x: 140, y: 500, w: 440, h: 72 };
const JOIN_CONFIRM_RECT = { x: 210, y: 640, w: 300, h: 64 };
const START_RECT = { x: 210, y: 900, w: 300, h: 64 };
const BACK_RECT = { x: 24, y: 24, w: 120, h: 48 };
function defaultServerUrl() {
  if (typeof location !== 'undefined' && location.hostname) {
    return 'ws://' + location.hostname + ':5174';
  }
  return 'ws://localhost:5174';
}
class MpRoomScene {
  constructor(game) {
    this.game = game;
    this.view = 'menu';
    this.name = '';
    this.serverUrl = defaultServerUrl();
    this.roomId = '';
    this.hostName = '';
    this.guestName = '';
    this.error = '';
    this.countdown = 0;
    this.enterCode = '';
    this.busy = false;
  }
  enter(params) {
    this.params = params || {};
    this.name = this.params.name || net.name || '';
    this.view = this.params.fromGame ? 'room' : 'menu';
    this.hostName = net.playerIndex === 1 ? this.name : net.peerName || '';
    this.guestName = net.playerIndex === 1 ? net.peerName || '' : this.name;
    this.roomId = net.roomId || '';
    this.error = '';
    this.countdown = 0;
    this.busy = false;
    net.onMessage = (msg) => this.handleMessage(msg);
    net.onClose = () => {
      if (this.view !== 'menu') {
        this.error = '连接已断开';
      }
    };
```

```
}
leave() {
  net.onMessage = null;
  net.onClose = null;
  net.close();
}
update(dt) {
  if (this.countdown > 0) {
    this.countdown -= dt;
    if (this.countdown <= 0) {
      this.countdown = 0;
      this.game.go('difficulty', {
        coop: true,
        pk: true,
        roomId: net.roomId,
        playerIndex: net.playerIndex,
        seed: net.seed,
        hostName: this.hostName,
        guestName: this.guestName,
      });
    }
    return;
  }
  const tap = this.game.input.consumeTap();
  if (!tap) return;
  if (this.inRect(tap, BACK_RECT)) {
    this.leave();
    this.game.go('title');
    return;
  }
  if (this.view === 'menu') {
    if (this.inRect(tap, CREATE_RECT)) {
      this.createRoom();
      return;
    }
    if (this.inRect(tap, JOIN_RECT)) {
      this.view = 'enterCode';
      this.enterCode = '';
      return;
    }
    if (this.inRect(tap, SERVER_RECT)) {
      this.openText('服务器地址', this.serverUrl, (v) => { this.serverUrl = v; });
    }
    return;
  }
  if (this.view === 'enterCode') {
    if (this.inRect(tap, ROOMCODE_RECT)) {
      this.openText('房间号(10位)', this.enterCode, (v) => { this.enterCode = v; });
      return;
    }
    if (this.inRect(tap, JOIN_CONFIRM_RECT)) {
      this.joinRoom();
    }
    return;
  }
```

```
  if (this.view === 'room') {
    if (net.playerIndex === 1 && this.guestName && this.inRect(tap, START_RECT)) {
      net.start();
    }
  }
}
createRoom() {
  if (this.busy) return;
  this.busy = true;
  this.error = '';
  net.connect(this.serverUrl)
    .then(() => {
      net.create(this.name);
    })
    .catch(() => {
      this.error = '无法连接服务器';
      this.busy = false;
    });
}
joinRoom() {
  if (this.busy) return;
  if (!/^\d{10}$/.test(this.enterCode)) {
    this.error = '请输入10位房间号';
    return;
  }
  this.busy = true;
  this.error = '';
  net.connect(this.serverUrl)
    .then(() => {
      net.join(this.enterCode, this.name);
    })
    .catch(() => {
      this.error = '无法连接服务器';
      this.busy = false;
    });
}
openText(title, value, done) {
  platform.showKeyboard({
    defaultValue: value,
    maxLength: 20,
    confirmType: 'done',
    onConfirm: (res) => {
      done((res && res.value) || '');
      platform.hideKeyboard();
    },
    onComplete: (res) => {
      const v = res && res.value !== undefined ? res.value : '';
      if (v) done(v);
      platform.hideKeyboard();
    },
  });
  void title;
}
handleMessage(msg) {
  if (msg.type === 'created') {
```

```
      this.roomId = msg.roomId;
      this.hostName = this.name;
      this.view = 'room';
      this.busy = false;
    } else if (msg.type === 'joined') {
      this.roomId = msg.roomId;
      this.hostName = msg.hostName;
      this.guestName = this.name;
      this.view = 'room';
      this.busy = false;
    } else if (msg.type === 'peerJoined') {
      this.guestName = msg.name;
    } else if (msg.type === 'start') {
      this.hostName = msg.hostName;
      this.guestName = msg.guestName;
      this.countdown = 3;
    } else if (msg.type === 'joinFailed') {
      this.error = msg.reason || '加入失败';
      this.busy = false;
    } else if (msg.type === 'peerLeft') {
      this.error = '对方已离开房间';
      this.leave();
      this.game.go('title');
    } else if (msg.type === 'gameover') {
      this.countdown = 0;
      this.view = 'room';
    }
  }
  inRect(point, rect) {
    return point.x >= rect.x && point.x <= rect.x + rect.w && point.y >= rect.y && point.y <= rect.y + re
ct.h;
  }
  render(ctx) {
    const sky = ctx.createLinearGradient(0, 0, 0, 1280);
    sky.addColorStop(0, '#0a1220');
    sky.addColorStop(0.6, '#1a2540');
    sky.addColorStop(1, '#0a101c');
    ctx.fillStyle = sky;
    ctx.fillRect(0, 0, 720, 1280);
    if (this.view === 'menu') {
      drawInkTitle(ctx, 'Local PK', 360, 200, 46);
      ctx.textAlign = 'center';
      ctx.fillStyle = 'rgba(220,232,255,0.8)';
      ctx.font = '20px sans-serif';
      ctx.fillText(`玩家：${this.name}`, 360, 300);
      drawButton(ctx, CREATE_RECT, 'Create Room', { font: 'bold 26px sans-serif', accent: 'rgba(140,200,2
55,0.9)' });
      drawButton(ctx, JOIN_RECT, 'Join Room', { font: 'bold 26px sans-serif', accent: 'rgba(255,160,120,0
.9)' });
      drawButton(ctx, SERVER_RECT, `服务器：${this.serverUrl}`, { font: 'bold 18px sans-serif' });
      if (this.busy) {
        ctx.fillStyle = 'rgba(255,255,255,0.7)';
        ctx.font = '20px sans-serif';
        ctx.textAlign = 'center';
        ctx.fillText('连接中…', 360, 730);
```

```
    }
  } else if (this.view === 'enterCode') {
    drawInkTitle(ctx, 'Join Room', 360, 220, 46);
    drawPanel(ctx, { x: 120, y: 480, w: 480, h: 110 });
    ctx.textAlign = 'center';
    ctx.fillStyle = this.enterCode ? '#f2f6ff' : 'rgba(220,232,255,0.45)';
    ctx.font = 'bold 30px sans-serif';
    ctx.fillText(this.enterCode || '点击输入10位房间号', 360, 542);
    drawButton(ctx, JOIN_CONFIRM_RECT, '加入', { font: 'bold 26px sans-serif' });
  } else {
    drawInkTitle(ctx, '房间', 360, 180, 52);
    drawPanel(ctx, { x: 110, y: 300, w: 500, h: 420 });
    ctx.textAlign = 'center';
    ctx.fillStyle = '#ffe9b8';
    ctx.font = 'bold 26px sans-serif';
    ctx.fillText(`房间号：${this.roomId}`, 360, 360);
    ctx.fillStyle = '#f2f6ff';
    ctx.font = 'bold 24px sans-serif';
    ctx.fillText(`player1：${this.hostName || '…'}`, 360, 430);
    ctx.fillText(`player2：${this.guestName || '等待玩家2加入…'}`, 360, 480);
    if (this.busy) {
      ctx.fillStyle = 'rgba(255,255,255,0.7)';
      ctx.font = '20px sans-serif';
      ctx.fillText('连接中…', 360, 540);
    } else if (net.playerIndex === 1) {
      if (this.guestName) {
        drawButton(ctx, START_RECT, 'Start', { font: 'bold 28px sans-serif', pulse: 0.15 + 0.1 * Math.s
in(this.game.time || 0) });
      } else {
        ctx.fillStyle = 'rgba(220,232,255,0.7)';
        ctx.font = '20px sans-serif';
        ctx.fillText('等待玩家2加入…', 360, 560);
      }
    } else {
      ctx.fillStyle = 'rgba(220,232,255,0.7)';
      ctx.font = '20px sans-serif';
      ctx.fillText('等待房主开始…', 360, 560);
    }
  }
  if (this.error) {
    ctx.fillStyle = '#ffb3a0';
    ctx.font = '18px sans-serif';
    ctx.textAlign = 'center';
    ctx.fillText(this.error, 360, 760);
  }
  drawButton(ctx, BACK_RECT, '返回', { font: 'bold 20px sans-serif' });
  if (this.countdown > 0) {
    ctx.fillStyle = 'rgba(4,8,18,0.72)';
    ctx.fillRect(0, 0, 720, 1280);
    ctx.fillStyle = '#fff3cf';
    ctx.font = 'bold 64px sans-serif';
    ctx.textAlign = 'center';
    ctx.fillText(String(Math.max(1, Math.ceil(this.countdown))), 360, 640);
    ctx.font = 'bold 22px sans-serif';
    ctx.fillText('即将开始游戏…', 360, 720);
```

```
    }
    ctx.textAlign = 'left';
  }
}
module.exports = MpRoomScene;
const { DIFFICULTIES, ORDER } = require('../config/difficulty.js');
const { drawButton, drawPanel, drawInkTitle } = require('../ui/menu.js');
const { clamp } = require('../utils/math.js');
const BACK_RECT = { x: 24, y: 24, w: 120, h: 48 };
const START_RECT = { x: 210, y: 1120, w: 300, h: 64 };
const DIFF_RECTS = ORDER.map((id, index) => ({
  id,
  rect: { x: 60 + index * 150, y: 300, w: 140, h: 64 },
}));
const STAGE_RECTS = [1, 2, 3, 4].map((stage) => ({
  stage,
  rect: { x: 150 + (stage - 1) * 105, y: 440, w: 90, h: 64 },
}));
const LV_MINUS = { x: 180, y: 580, w: 70, h: 56 };
const LV_PLUS = { x: 470, y: 580, w: 70, h: 56 };
const BOMB_MINUS = { x: 180, y: 700, w: 70, h: 56 };
const BOMB_PLUS = { x: 470, y: 700, w: 70, h: 56 };
const INV_RECT = { x: 210, y: 830, w: 300, h: 64 };
class DebugStageScene {
  constructor(game) {
    this.game = game;
    this.time = 0;
  }
  enter() {
    this.difficulty = 'easy';
    this.stage = 1;
    this.level = 1;
    this.bombs = 3;
    this.invincible = false;
    this.time = 0;
    this.game.audio.playTitleBgm();
  }
  stageMax() {
    return DIFFICULTIES[this.difficulty].stageCount;
  }
  update(dt) {
    this.time += dt;
    const tap = this.game.input.consumeTap();
    if (!tap) return;
    if (this.inRect(tap, BACK_RECT)) {
      this.game.audio.pickup();
      this.game.go('title');
      return;
    }
    if (this.inRect(tap, START_RECT)) {
      this.game.audio.pickup();
      const maxHp = 100 + 12 * (this.level - 1);
      this.game.go('stage', {
        mode: 'new',
        difficulty: this.difficulty,
```

```
    stage: this.stage,
    invincible: this.invincible,
    noSave: true,
    debugStage: true,
    progress: {
      level: this.level,
      exp: 0,
      expTotal: 0,
      bombs: this.bombs,
      lives: 3,
      invincible: this.invincible,
      hp: maxHp,
      maxHp,
      score: 0,
      claimedExtras: [],
    },
  });
  return;
}
for (const diff of DIFF_RECTS) {
  if (this.inRect(tap, diff.rect)) {
    this.difficulty = diff.id;
    this.stage = Math.min(this.stage, this.stageMax());
    this.game.audio.pickup();
    return;
  }
}
for (const st of STAGE_RECTS) {
  if (st.stage <= this.stageMax() && this.inRect(tap, st.rect)) {
    this.stage = st.stage;
    this.game.audio.pickup();
    return;
  }
}
if (this.inRect(tap, LV_MINUS)) {
  this.level = clamp(this.level - 1, 1, 26);
  this.game.audio.pickup();
  return;
}
if (this.inRect(tap, LV_PLUS)) {
  this.level = clamp(this.level + 1, 1, 26);
  this.game.audio.pickup();
  return;
}
if (this.inRect(tap, BOMB_MINUS)) {
  this.bombs = clamp(this.bombs - 1, 0, 5);
  this.game.audio.pickup();
  return;
}
if (this.inRect(tap, BOMB_PLUS)) {
  this.bombs = clamp(this.bombs + 1, 0, 5);
  this.game.audio.pickup();
  return;
}
if (this.inRect(tap, INV_RECT)) {
```

```
      this.invincible = !this.invincible;
      this.game.audio.pickup();
    }
  }
  inRect(point, rect) {
    return point.x >= rect.x && point.x <= rect.x + rect.w && point.y >= rect.y && point.y <= rect.y + re
ct.h;
  }
  render(ctx) {
    const sky = ctx.createLinearGradient(0, 0, 0, 1280);
    sky.addColorStop(0, '#0a1220');
    sky.addColorStop(0.6, '#16233a');
    sky.addColorStop(1, '#0a101c');
    ctx.fillStyle = sky;
    ctx.fillRect(0, 0, 720, 1280);
    drawInkTitle(ctx, '内测跳关', 360, 150, 54);
    ctx.textAlign = 'center';
    ctx.fillStyle = 'rgba(220,232,255,0.75)';
    ctx.font = '18px sans-serif';
    ctx.fillText('选择难度 / 关卡 / 初始状态', 360, 215);
    ctx.fillStyle = '#ffe9b8';
    ctx.font = 'bold 22px sans-serif';
    ctx.fillText('难度', 360, 275);
    for (const diff of DIFF_RECTS) {
      drawButton(ctx, diff.rect, DIFFICULTIES[diff.id].label, {
        font: 'bold 20px sans-serif',
        active: this.difficulty === diff.id,
        accent: this.difficulty === diff.id ? 'rgba(255,211,107,0.9)' : undefined,
      });
    }
    ctx.fillStyle = '#ffe9b8';
    ctx.font = 'bold 22px sans-serif';
    ctx.fillText('关卡', 360, 415);
    for (const st of STAGE_RECTS) {
      const locked = st.stage > this.stageMax();
      drawButton(ctx, st.rect, String(st.stage), {
        font: 'bold 22px sans-serif',
        active: !locked && this.stage === st.stage,
        fill: locked ? 'rgba(20,28,44,0.35)' : undefined,
        frame: locked ? 'rgba(220,232,255,0.25)' : undefined,
        color: locked ? 'rgba(220,232,255,0.35)' : undefined,
      });
    }
    this.drawStepper(ctx, '自机等级', this.level, 26, LV_MINUS, LV_PLUS, 555);
    this.drawStepper(ctx, '炸弹数', this.bombs, 5, BOMB_MINUS, BOMB_PLUS, 675);
    drawButton(ctx, INV_RECT, `无敌模式：${this.invincible ? '开' : '关'}`, {
      font: 'bold 22px sans-serif',
      active: this.invincible,
      accent: this.invincible ? 'rgba(140,255,190,0.9)' : undefined,
      glowColor: '#7dffc0',
    });
    ctx.fillStyle = 'rgba(255,179,161,0.9)';
    ctx.font = '16px sans-serif';
    ctx.textAlign = 'center';
    ctx.fillText('此模式无法保存进度', 360, 925);
```

```
    if (this.invincible) {
      ctx.fillStyle = 'rgba(157,255,200,0.85)';
      ctx.font = '16px sans-serif';
      ctx.fillText('炸弹无限 • 残机无限', 360, 945);
    }
    drawButton(ctx, START_RECT, '开始作战', {
      font: 'bold 26px sans-serif',
      pulse: 0.15 + 0.1 * Math.sin(this.time * 1.8),
      accent: 'rgba(255,120,140,0.9)',
    });
    drawButton(ctx, BACK_RECT, '返回', { font: 'bold 20px sans-serif' });
  }
  drawStepper(ctx, label, value, max, minusRect, plusRect, labelY) {
    ctx.fillStyle = '#ffe9b8';
    ctx.font = 'bold 22px sans-serif';
    ctx.textAlign = 'center';
    ctx.fillText(label, 360, labelY);
    drawButton(ctx, minusRect, '-', { font: 'bold 26px sans-serif' });
    drawButton(ctx, plusRect, '+', { font: 'bold 26px sans-serif' });
    ctx.fillStyle = '#fff3d6';
    ctx.font = 'bold 30px sans-serif';
    ctx.fillText(`${value} / ${max}`, 360, labelY + 52);
  }
}
module.exports = DebugStageScene;
class Multiplayer {
  constructor() {
    this.conn = null;
    this.sendRaw = null;
    this.roomId = null;
    this.playerIndex = 0;
    this.name = '';
    this.peerName = '';
    this.seed = 0;
    this.url = '';
    this.onMessage = null;
    this.onClose = null;
    this.clockOffsetMs = 0;
    this.smoothedRttMs = 60;
    this._pingTimer = null;
    this._pingCount = 0;
    this._pingSamples = [];
    this._rttSamples = [];
  }
  connect(url) {
    return new Promise((resolve, reject) => {
      this.url = url;
      let settled = false;
      const fail = (e) => {
        if (settled) return;
        settled = true;
        clearTimeout(timer);
        if (this.onClose) this.onClose();
        reject(e || new Error('连接失败'));
      };
```

```
    const done = () => {
      if (settled) return;
      settled = true;
      clearTimeout(timer);
      resolve();
    };
    const timer = setTimeout(() => fail(new Error('连接超时')), 6000);
    const wxApi = typeof wx !== 'undefined' ? wx : null;
    if (wxApi && wxApi.connectSocket) {
      const task = wxApi.connectSocket({ url, fail: (e) => fail(e) });
      task.onOpen(() => {
        this.conn = task;
        this.sendRaw = (s) => task.send({ data: s });
        done();
        this._beginClockSync();
      });
      task.onMessage((res) => this.dispatch(res.data));
      task.onClose(() => fail(new Error('连接已断开')));
      task.onError((e) => fail(e));
      return;
    }
    if (typeof WebSocket === 'undefined') {
      fail(new Error('WebSocket 不可用'));
      return;
    }
    const ws = new WebSocket(url);
    ws.onopen = () => {
      this.conn = ws;
      this.sendRaw = (s) => ws.send(s);
      done();
      this._beginClockSync();
    };
    ws.onmessage = (e) => this.dispatch(e.data);
    ws.onclose = () => fail(new Error('连接已断开'));
    ws.onerror = () => fail(new Error('连接失败'));
  });
}
_beginClockSync() {
  this._pingCount = 0;
  this._pingSamples = [];
  this._rttSamples = [];
  const sample = () => {
    this._pingCount += 1;
    this.send({ type: 'ping', t: Date.now() });
    this._pingTimer = setTimeout(sample, this._pingCount < 3 ? 200 : 1000);
  };
  sample();
}
send(obj) {
  if (this.sendRaw) this.sendRaw(JSON.stringify(obj));
}
dispatch(raw) {
  let msg;
  try { msg = JSON.parse(raw); } catch (e) { return; }
  if (msg.type === 'pong') {
```

```
    const sentAt = typeof msg.t === 'number' ? msg.t : 0;
    const serverTime = typeof msg.serverTime === 'number' ? msg.serverTime : Date.now();
    if (sentAt > 0) {
      const recvAt = Date.now();
      const offset = serverTime - (sentAt + recvAt) / 2;
      const rtt = recvAt - sentAt;
      this._pingSamples.push(offset);
      this._rttSamples.push(rtt);
      if (this._pingSamples.length > 5) this._pingSamples.shift();
      if (this._rttSamples.length > 5) this._rttSamples.shift();
      let sum = 0;
      for (const s of this._pingSamples) sum += s;
      this.clockOffsetMs = Math.round(sum / this._pingSamples.length);
      let rttSum = 0;
      for (const r of this._rttSamples) rttSum += r;
      this.smoothedRttMs = Math.max(4, Math.round(rttSum / this._rttSamples.length));
    }
  } else if (msg.type === 'created') {
    this.roomId = msg.roomId;
    this.playerIndex = msg.playerIndex || 1;
  } else if (msg.type === 'joined') {
    this.roomId = msg.roomId;
    this.playerIndex = msg.playerIndex || 2;
    this.peerName = msg.hostName || '';
  } else if (msg.type === 'peerJoined') {
    this.peerName = msg.name || '';
  } else if (msg.type === 'pkForfeit') {
    if (msg.host) this.playerIndex = 1;
  } else if (msg.type === 'start' || msg.type === 'difficulty') {
    this.seed = msg.seed || 0;
  }
  if (this.onMessage) this.onMessage(msg);
}
create(name) {
  this.name = name || '';
  this.send({ type: 'create', name: this.name });
}
join(roomId, name) {
  this.name = name || '';
  this.send({ type: 'join', roomId: String(roomId || ''), name: this.name });
}
start() {
  this.send({ type: 'start' });
}
selectDifficulty(difficulty) {
  this.send({ type: 'selectDifficulty', difficulty });
}
sendIntent(frame, move, bomb, stageId) {
  this.send({
    type: 'input',
    frame: Number(frame) || 0,
    move: move || null,
    bomb: !!bomb,
    stage: stageId || 0,
  });
```

```
  }
  sendPkForfeit() {
      this.send({ type: 'pkForfeit' });
    }
  sendPkState(state) {
      this.send({
        type: 'pkState',
        score: Number(state.score) || 0,
        stage: Number(state.stage) || 1,
        lives: Number(state.lives) || 0,
        hp: Number(state.hp) || 0,
        maxHp: Number(state.maxHp) || 100,
        over: !!state.over,
      });
    }
    sendPkFinished(score, stage, cleared) {
      this.send({ type: 'pkFinished', score: Number(score) || 0, stage: Number(stage) || 1, cleared: !!cl
eared });
    }
    sendPkWatch(on) {
      this.send({ type: 'pkWatchOn', on: !!on });
    }
    sendPkFrame(data) {
      this.send({ type: 'pkFrame', data: String(data || '') });
    }
  sendInput(move) {
    this.sendIntent(0, move, false, 0);
  }
  gameover() {
    this.send({ type: 'gameover' });
  }
  close() {
    if (this._pingTimer) {
      clearTimeout(this._pingTimer);
      this._pingTimer = null;
    }
    this._pingCount = 0;
    this._pingSamples = [];
    this._rttSamples = [];
    this.clockOffsetMs = 0;
    this.smoothedRttMs = 60;
    if (this.conn) {
      try {
        if (typeof this.conn.close === 'function') this.conn.close();
        else this.conn.close({});
      } catch (e) {}
    }
    this.conn = null;
    this.sendRaw = null;
    this.roomId = null;
    this.playerIndex = 0;
    this.peerName = '';
  }
}
module.exports = new Multiplayer();
```